AI-Powered Resume Screening and Candidate Ranking System with Full Stack Web Development.

A Main project Submitted in Partial fulfilment of the Requirements for the award of the Degree of

BACHELOR OF TECHNOLOGY
In
CSE – ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Submitted by

G. Subhash Babu
22F01A4217

P. Devanand
22F01A4244

G. Prateet Sai
22F01A4219

B. Venkaiah Naidu
22F01A4204

Under the guidance of

Mr.V.Prasad
Associate Professor

DEPT. OF CSE – ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

ST. ANN'S COLLEGE OF ENGINEERING & TECHNOLOGY
AUTONOMOUS

[Approved by A.I.C.T.E. & Permanently Affiliated to JNTUK- Kakinada]

[Accredited by NBA (Four Times) and permanently accredited by IE[I]]

[Accredited by NAAC with 'A' Grade (1st, 2nd and 3rd cycles) &

Awarded 'A' Grade by AP State Govt.]

[Recognized by UGC u/s 2(f) & 12(B) of the UGC Act 1956]

Chirala – 523187, Bapatla District. URL: **www. sacet.ac.in**

ST. ANN'S COLLEGE OF ENGINEERING & TECHNOLOGY
AUTONOMOUS
CHIRALA

DEPT. OF CSE – ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

CERTIFICATE

This is to certify that the project entitled **"AI-Powered Resume Screening and Candidate Ranking System with Full Stack Web Development"** is being submitted by **G.Subhash Babu (22F01A4217)** along with his batchmates **P.Devanand (22F01A4244)**, **G.Prateet Sai (22F01A4219), B.Venkaiah Naidu (22F01A4204)** in partial fulfillment of the requirements for the award of the degree of **B.Tech in CSE – Artificial Intelligence & Machine Learning**, **Jawaharlal Nehru Technological University, Kakinada** is a record of **bonafide work** carried out by the students under my guidance and supervision The results presented in this project have been **verified** and are found to be **satisfactory.**

Internal Guide
Mr. V. Prasad

Head of the Department
Dr. C. Hari Kishan

External Examiner

DECLARATION

I**, G. SUBHASH BABU** as a student of B. Tech Program, **Reg. No. 22F01A4217** with batchmates P. DEVANAND – 22F01A4244, G. PRATEET SAI - 22F01A4219, B. VENKAIAH NAIDU - 22F01A4204 of the Department of CSE - AI&ML in St. Ann's College of Engineering and Technology, Chirala do here by declare that I have completed the mandatory project from December 2025 in DataValley under the Faculty Guidance of Mr. V. PRASAD, Department of CSE (AI&ML), St. Ann's College of Engineering and Technology, Chirala.

G. SUBHASH BABU

22F01A4217

ACKNOWLEDGEMENT

The successful completion of any task is not possible without proper suggestion, guidance and environment. Combination of these three factors acts like backbone to my project titled **"AI-Powered Resume Screening and Candidate Ranking System with Full Stack Web Development"**.

We express my sincere thanks to our guide **Mr. V. Prasad** and also, Head of the Department of CSE - Artificial Intelligence & Machine Learning, **Dr. C. Hari Kishan** for their valuable suggestions during our course period, timely help, guidance and providing me with the most essential materials required for the completion of this work.

We are thankful to all teaching and non-teaching staff of the CSE - Artificial Intelligence & Machine Learning for the cooperation given for the successful completion of our project.

We would like to thank our Principal, **Dr. K. Jagadeesh Babu** for providing support and simulating environment. I would like to express our gratitude to the **Management** of **St. Ann's College of Engineering & Technology** for providing us a pleasant environment and excellent laboratory facilities.

GENI SUBHASH BABU (22F01A4217)
PARI DEVANAND (22F01A4244)
GUDURI PRATEET SAI (22F01A4219)
BELLAMKONDA VENKAIAH NAIDU (22F01A4204)

TABLE OF CONTENTS

	PAGE NO
CERTIFICATE	**I**
DECLARATION	**II**
ACKNOWLEDGEMENT	**III**
TABLE OF CONTENTS	**IV**
LIST OF FIGURES	**VI**
LIST OF TABLES	**VII**
LIST OF ACRONYMS	**VIII**
ABSTRACT	**IX**

CHAPTER 1: INTRODUCTION **10**

1.1 PROBLEM STATEMENT 11

1.2 EXISTING SYSTEM 12

1.3 PROPOSED SYSTEM 14

1.4 ORGANIZATION OF THESIS 16

CHAPTER 2: LITERATURE SURVEY **18**

2.1 BASIC CONCEPTS 18

2.2 PROJECT RELATED WORK 22

CHAPTER 3: FEASIBILITY STUDY **26**

3.1 TECHNICAL FEASIBILITY 26

3.2 OPERATIONAL FEASIBILITY 30

3.3 ECONOMIC FEASIBILITY 32

CHAPTER 4: SOFTWARE REQUIREMENT SPECIFICATIONS **34**

4.1 FUNCTIONAL REQUIREMENTS 34

4.2 NON-FUNCTIONAL REQUIREMENTS 35

4.2.1 PERFORMANCE REQUIREMENTS 35

4.2.2 HARDWARE REQUIREMENTS 35

4.2.3 SOFTWARE REQUIREMENTS 36

4.2.4 INTRODUCTION TO JAVA / PYTHON 38

CHAPTER 5: SYSTEM MODELING **40**
5.1 REQUIREMNETS MODELING 40
5.1.1 CLASS DIAGRAMS 41
5.1.2 USE CASE DIAGRAMS 42
5.1.3 SEQUENCE DIAGRAMS 42
5.1.4 COLLABORATION DIAGRAMS 43
5.1.5 ACTIVITY DIAGRAMS 44
5.1.6 DEPLOYMENT DIAGRAMS 45
5.1.7 MODULES DESCRIPTION 46
5.2 DESIGN MODELING 48
5.2.1 SYSTEM ARCHITECTURE 48

CHAPTER 6: SYSTEM IMPLEMENTATION **50**
6.1 DEVELOPMENT ENVIRONMENT AND TOOLS 51
6.2 CODING 52
6.2.1 SOURCE CODE 53

CHAPTER 7: TESTING **60**
7.1 INTRODUCTION TO TESTING 60
7.1.1 TESTING STRATEGIES 61
7.1.2 UNIT TESTING 61
7.1.3 INTEGRATION TESTING 61
7.1.4 SYSTEM TESTING 62
7.1.5 TEST CASES 62

CHAPTER 8: RESULTS **63**
8.1 SCREENS AND REPORTS 64
8.2 USER MANUAL 66

CHAPTER 9: LIMITATIONS **71**
CHAPTER 10: CONCLUSION & FUTURE WORK **72**
CHAPTER 11: REFERENCES **74**

LIST OF FIGURES

FIG. NUMBER	FIG. NAME	PAGE NUMBER
Fig 1	System Architecture of Resume Screening System	11
Fig 2	Resume Processing Pipeline (NLP Workflow)	13
Fig 3	Proposed System Workflow	15
Fig 4	TF-IDF Vectorization and Similarity Process	25
Fig 5	Candidate Ranking Flow using Cosine Similarity	27
Fig 6	Class Diagram	35
Fig 7	Use Case Diagram	36
Fig 8	Sequence Diagram	37
Fig 9	Collaboration Diagram	38
Fig 10	Activity Diagram	39
Fig 11	Deployment Diagram	40
Fig 12	System Architecture	43
Fig 13	Resume Parsing and Feature Extraction Process	44
Fig 14	System Implementation Workflow	45
Fig 15	Performance Evaluation and Accuracy Comparison	46
Fig 16	User Interface of the Recruitment Web Application	53
Fig 17	Candidate Ranking Output Visualization	54

LIST OF TABLES

TABLE NUMBER	TABLE NAME	PAGE NUMBER
Table 1	Literature Review	24-25

LIST OF ACRONYMS

AI – Artificial Intelligence

ML – Machine Learning

NLP – Natural Language Processing

TF-IDF – Term Frequency-Inverse Document Frequency

API – Application Programming Interface

UI – User Interface

UX – User Experience

PDF – Portable Document Format

CSV – Comma Separated Values

DB – Database

SQL – Structured Query Language

NoSQL – Not Only SQL

IDE – Integrated Development Environment

UML – Unified Modeling Language

HTTP – Hyper Text Transfer Protocol

JSON – JavaScript Object Notation

ATS – Applicant Tracking System

ABSTRACT

This project presents the design and implementation of an **AI-Powered Resume Screening and Candidate Ranking System** aimed at automating and optimizing the recruitment process through intelligent data analysis. The system integrates **Natural Language Processing (NLP)** and **Machine Learning techniques** to extract, analyze, and evaluate candidate information from unstructured resume data. A structured pipeline is developed for **resume parsing, text preprocessing, feature extraction using TF-IDF vectorization**, and **semantic similarity analysis** to accurately match candidate profiles with job descriptions.

To enhance ranking accuracy, supervised learning models are employed to score and rank candidates based on skill relevance, experience alignment, and contextual keyword matching. The system is implemented as a **full stack web application**, incorporating a user-friendly interface, backend processing, secure authentication, and database integration to ensure scalability and usability. Comparative evaluation of ranking results demonstrates improved consistency, reduced manual bias, and significant time efficiency compared to traditional screening methods

The proposed system provides a **robust, scalable, and practical recruitment solution**, enabling organizations to streamline hiring workflows while maintaining accuracy and fairness in candidate selection

Keywords:

AI-Powered Recruitment, Resume Screening, Candidate Ranking, Natural Language Processing (NLP), TF-IDF, Semantic Similarity, Machine Learning, Full Stack Web Development, Automated Hiring System

CHAPTER 1: INTRODUCTION

In today's competitive job market, organizations receive a large volume of job applications for a single position, making manual resume screening a time-consuming and error-prone process. Recruiters are often required to review hundreds or even thousands of resumes within limited time constraints, which increases the risk of overlooking qualified candidates and introduces subjectivity into the hiring process. This challenge highlights the need for an automated, efficient, and intelligent system to assist recruiters in identifying suitable candidates accurately and consistently.

An **AI-Powered Resume Screening and Candidate Ranking System** aims to address these challenges by leveraging machine learning and natural language processing techniques to analyze resumes automatically. The primary objective of this system is to extract relevant information from resumes, evaluate candidate skills and experience, and rank applicants based on their relevance to a given job description. By automating the initial screening process, organizations can significantly reduce recruitment time while improving the quality of candidate shortlisting.

Traditional resume screening methods rely heavily on keyword matching and manual evaluation, which often fail to capture the semantic meaning of candidate profiles. Such approaches may miss qualified candidates due to variations in resume formats, wording, or skill representation. In contrast, AI-based systems can understand contextual relationships between skills, experience, and job requirements, leading to more accurate and unbiased candidate evaluation

The proposed system utilizes machine learning algorithms and text vectorization techniques to transform unstructured resume data into meaningful numerical representations. By comparing these representations with job requirements, the system computes relevance scores and generates a ranked list of candidates. This ranking assists recruiters in making informed decisions while minimizing human bias and repetitive effort. Additionally, the system supports scalability, allowing it to handle large volumes of resumes efficiently.

Overall, this project focuses on developing a robust, intelligent, and scalable resume screening solution that enhances recruitment efficiency and decision-making. By integrating AI-driven analysis with automated candidate ranking, the proposed system offers a practical approach to modern hiring challenges and demonstrates the effective application of artificial intelligence in human resource management.

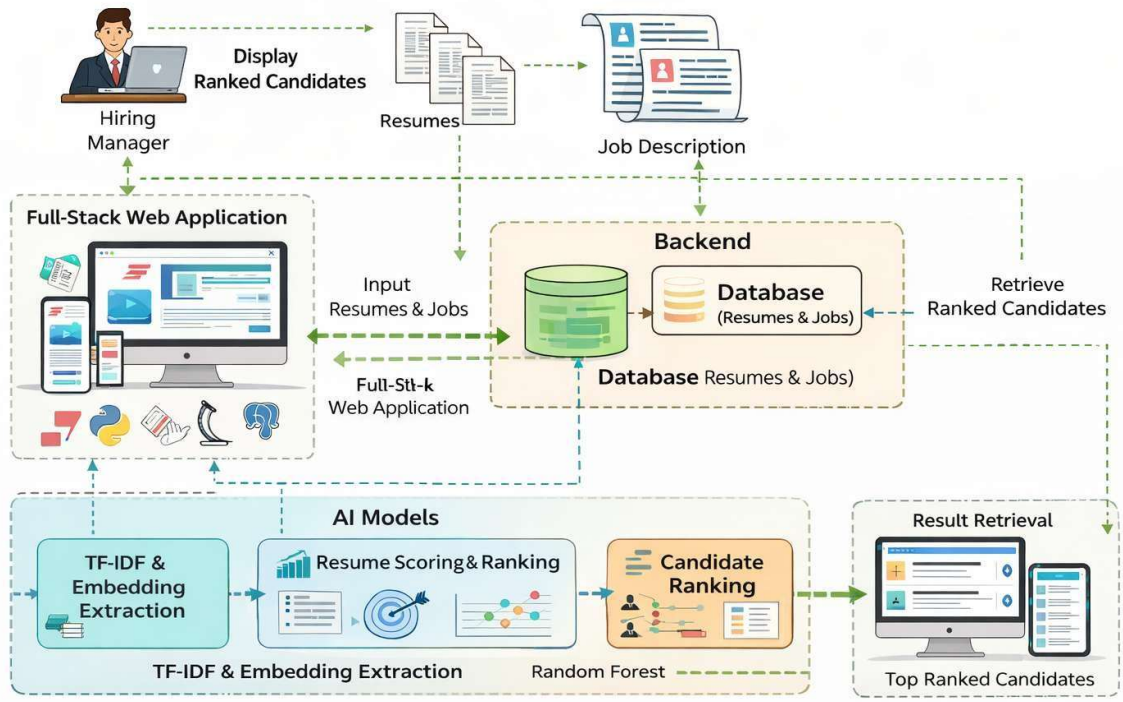


Fig 1. System Architecture for AI-Powered Resume Screening and Candidate Ranking with Full-Stack Web Development

1.1 PROBLEM STATEMENT

Resume screening is a critical process in modern recruitment systems, involving the analysis and evaluation of candidate resumes to identify suitable applicants for specific job roles. With the rapid growth of online job applications, recruiters are required to process **hundreds or thousands of resumes**, making manual screening time-consuming, inconsistent, and prone to human bias. Efficient resume evaluation is essential for ensuring fair candidate selection, faster hiring cycles, and improved talent matching.

Despite the availability of digital recruitment platforms, several challenges continue to affect the effectiveness of traditional resume screening methods:

1. **Unstructured Resume Data:** Resumes are submitted in varied formats and styles, making it difficult to extract and standardize relevant information such as skills, experience, and qualifications.
2. **Keyword Dependency:** Conventional screening systems rely heavily on exact keyword matching, often overlooking qualified candidates whose resumes use different terminology ctions.
3. **Recruiter Bias and Inconsistency:** Manual evaluation is subjective and may lead to biased or

inconsistent shortlisting decisions.

4. **Scalability Issues:** Handling large volumes of resumes efficiently is challenging, especially in high-volume recruitment scenarios.
5. **Poor Job–Candidate Matching:** Existing systems struggle to accurately measure semantic relevance between job descriptions and candidate profiles.

Given these challenges, there is a need for an **AI-powered resume screening and candidate ranking system** that can au Traditional tomatically analyze resumes, extract meaningful features, understand semantic context, and rank candidates based on their relevance to job requirements. Such a system should improve screening accuracy, reduce processing time, and assist recruiters in making data-driven and unbiased hiring decisions.

1.2 EXISTING SYSTEM

Traditional resume screening systems relied heavily on **manual evaluation** and **rule-based filtering mechanisms** implemented through Applicant Tracking Systems (ATS). These systems process resumes using predefined criteria such as keyword presence, years of experience, and educational qualifications. While effective for basic filtering, they lack intelligence and contextual understanding of candidate profiles.

With the introduction of machine learning, early automated resume screening systems employed **classical text-processing techniques** combined with traditional classifiers. These methods improved efficiency but remained limited in handling semantic variations and real-world resume diversity. Some notable existing approaches include:

- **Keyword-Based ATS Systems:**
  These systems filter resumes by matching predefined keywords with job descriptions. Although fast, they often reject qualified candidates due to vocabulary mismatch and rigid rules.
- **Bag-of-Words (BoW) and TF-IDF Models:**
  Used to represent resumes numerically based on term frequency. While effective for basic similarity scoring, they fail to capture contextual and semantic meaning.
- **Naïve Bayes and Support Vector Machines (SVM):**
  Applied for resume classification and job-role prediction. These models perform reasonably on structured data but struggle with unstructured resumes and nuanced skill descriptions.

- **Rule-Based Ranking Systems:**

  Assign scores based on fixed weightage for skills, experience and education.

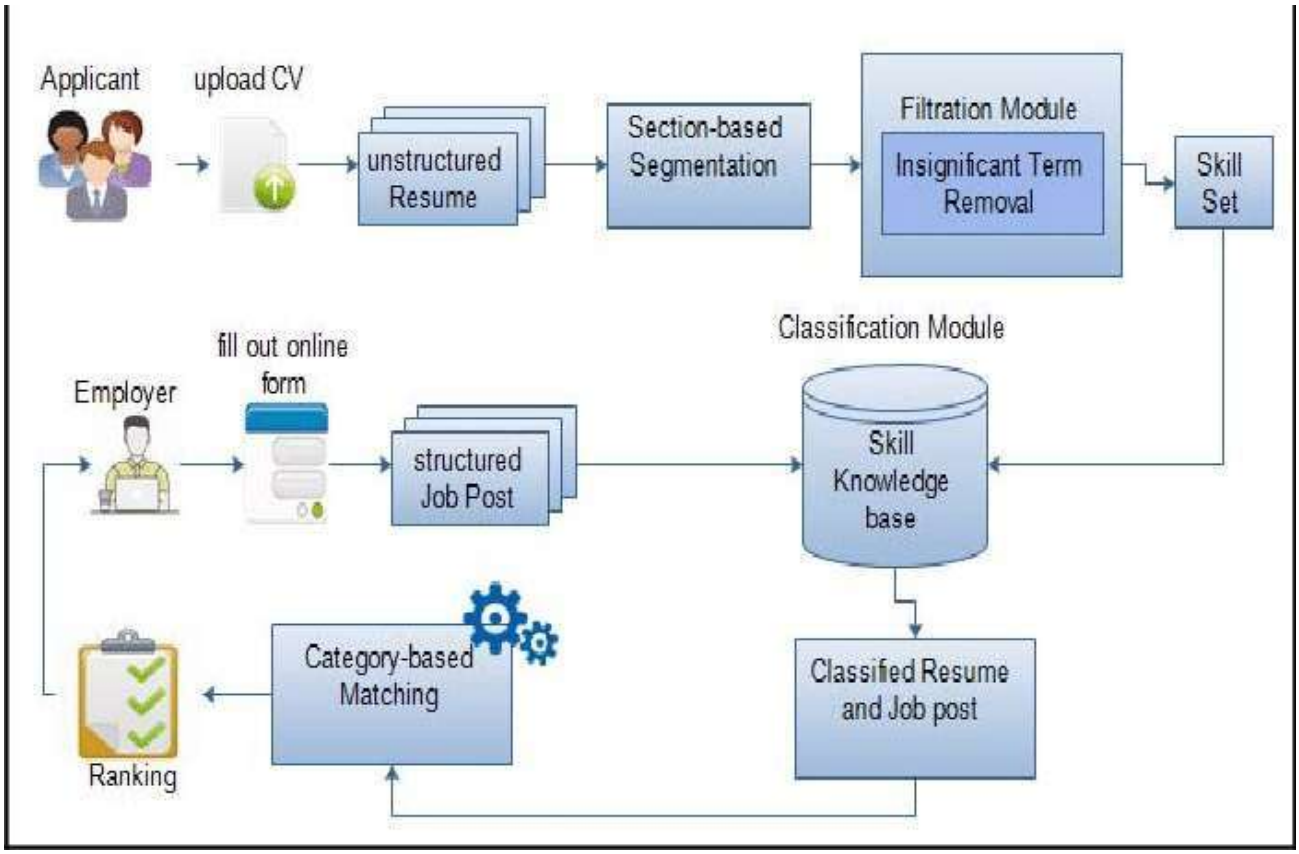


Fig 1.2: Resume Processing Pipeline (NLP Workflow)

Limitations of Existing Systems:

1. **High Dependence on Manual and Rule-Based Processing:**

   Traditional resume screening systems rely heavily on predefined rules and manual shortlisting, which makes the process time-consuming and inconsistent when handling large volumes of resumes.

2. **Lack of Semantic Understanding:**

   Most existing systems depend on exact keyword matching techniques, failing to understand semantic similarity between job descriptions and candidate skills. This leads to the rejection of qualified candidates using varied terminology.

3. **Rigid Single-Representation Models:**

   Conventional text-processing approaches represent resumes using a single feature space (e.g., keyword frequency or bag-of-words), limiting their ability to

capture contextual meaning, skill relationships, and experience relevance.

4. **Sensitivity to Resume Format Variations:**
   Differences in resume layouts, file formats, and writing styles negatively impact information extraction accuracy, resulting in incomplete or incorrect candidate profiling.

1.3 PROPOSED SYSTEM

To address the limitations of traditional resume screening methods, we propose an **AI-Powered Resume Screening and Candidate Ranking System** integrated with a **full-stack web application**. The proposed system adopts a **dual-analysis framework** that combines semantic text understanding with structured candidate evaluation to improve accuracy, scalability, and fairness in recruitment.

1. **Semantic Analysis Stream:**

2 Extracts meaningful information from resumes and job descriptions using natural language processing techniques.

3 Converts unstructured resume text into numerical representations using **TF-IDF and embedding-based feature extraction**.

4 Captures contextual and semantic similarity between candidate profiles and job requirements.

2. Candidate Scoring and Ranking Stream:

5 candidates based on extracted features such as skills, experience, and qualifications.

6 Applies machine learning models to compute relevance scores between resumes and job descriptions.

7 Ranks canEvaluates didates objectively based on overall suitability rather than isolated keywords

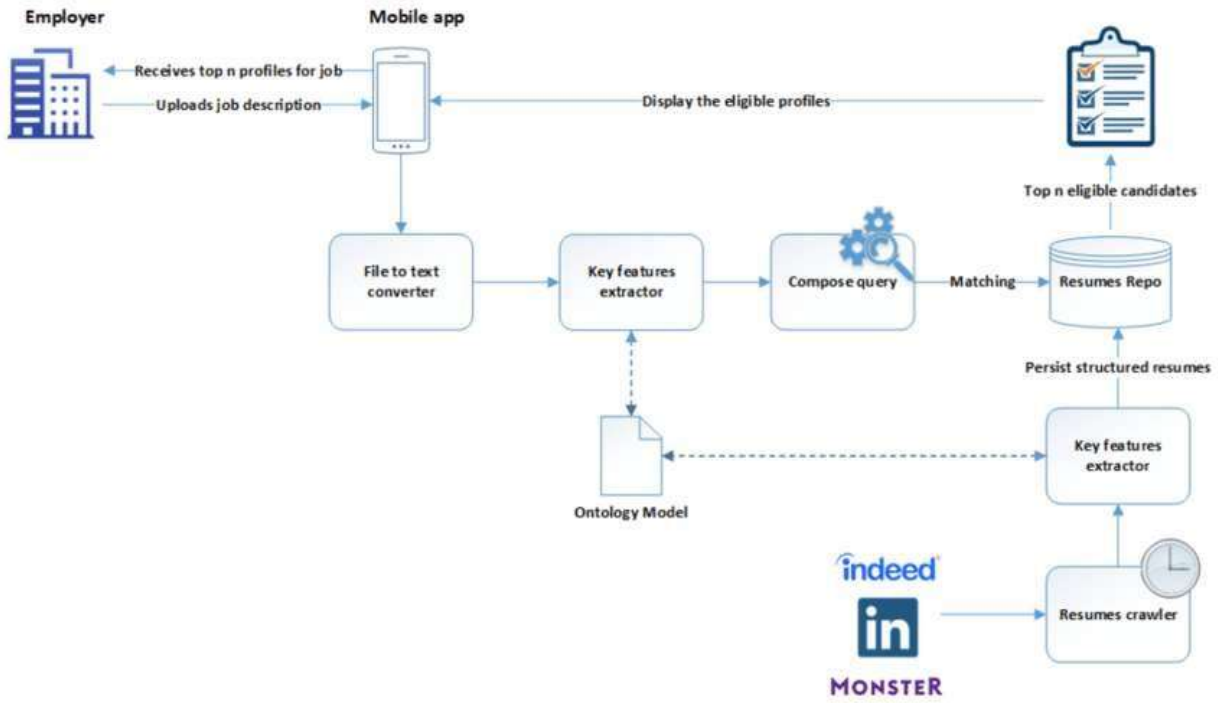


Fig 1.3: Proposed System

Key Features of the Proposed System:

- **SemanticFeatureExtraction**:
  Extracts meaningful information from resumes and job descriptions using NLP techniques, enabling accurate understanding of skills, experience, and qualifications beyond simple keyword matching.
- **Feature Fusion Strategy:**
  Combines multiple resume attributes such as skills, education, and experience into a unified representation, improving candidate–job relevance assessment.
- **Context-Aware Ranking Mechanism:**
  Evaluates candidates based on semantic similarity and relevance scores, ensuring accurate and unbiased ranking aligned with job requirements

- **Lightweight Machine Learning Models:**
  Utilizes computationally efficient models to reduce processing time while maintaining

high screening accuracy, making the system scalable for large resume volumes.

- **Real-Time Full-Stack Processing:**
  Integrates AI models with a full-stack web application to enable real-time resume upload, analysis, and ranked result visualization for recruiters.

Advantages Over Existing Systems:

1. **Improved Semantic Understanding:**
   The proposed system analyzes resumes and job descriptions semantically rather than relying on exact keyword matching, ensuring that relevant candidates are accurately identified even when different terminology is used..
2. **Higher Screening Accuracy:**
   By combining contextual text analysis with intelligent ranking mechanisms, the system provides more precise candidate shortlisting compared to traditional rule-based and single-feature screening approaches.
3. **Reduced Human Bias:**
   Automated scoring and ranking minimize subjective recruiter bias, leading to consistent and fair candidate evaluation.
4. **Lower Processing Time:**
   Efficient feature extraction and lightweight machine learning models significantly reduce resume screening time, enabling real-time analysis even for large-scale recruitment.

By integrating **natural language processing, machine learning–based ranking, and full-stack web technologies**, the proposed system delivers a robust, accurate, and scalable solution for automated resume screening and candidate selection.

7.1 ORGANIZATION OF THESIS is structured as follows:

- **Chapter 1: Introduction**

  This chapter introduces the concept of AI-powered resume screening and candidate ranking. It defines the problem of manual and rule-based recruitment systems, highlights their limitations such as bias, inefficiency, and scalability issues, and presents the motivation and objectives of the proposed system.

- **Chapter 2: Literature Review**

  This chapter reviews existing research and industry practices in automated recruitment systems. It covers traditional keyword-based screening, machine learning approaches, natural language processing techniques, and recent advancements such as semantic similarity, TF-IDF, cosine similarity, and learning-based ranking models used in resume analysis.

- **Chapter 3: System Architecture and Methodology**

  This chapter provides a detailed description of the proposed AI-powered resume screening system. It explains the overall system architecture, data flow, resume parsing mechanism, feature extraction using NLP techniques, similarity computation, candidate scoring, and ranking strategy. The technologies and algorithms used in system development are also discussed.

- **Chapter 4: Implementation and Experimental Results**

  This chapter presents the implementation details and execution of the system. It includes the dataset description, preprocessing steps, model execution workflow, and experimental results. Performance analysis is conducted based on ranking accuracy, relevance of shortlisted candidates, and comparison with traditional screening approaches.

- **Chapter 5: Conclusion and Future Work**

  This chapter concludes the thesis by summarizing the outcomes of the proposed system and its effectiveness in improving recruitment efficiency. It also discusses future enhancements such as integration of deep learning models, bias mitigation techniques, skill ontology expansion, and real-time deployment for enterprise-level recruitment platforms.

CHAPTER 2: LITERATURE SURVEY

2.1 BASIC CONCEPTS

AI-Powered Resume Screening is an emerging research area in artificial intelligence and natural language processing that focuses on automatically analyzing, filtering, and ranking candidate resumes based on their relevance to a given job description. It aims to reduce manual effort in recruitment by extracting meaningful information such as skills, experience, education, and certifications from unstructured resume data.

Fundamental Concepts in AI-Powered Resume Screening:

Resume Representation in Automated Screening:

AI-powered resume screening systems represent candidate information using different models to convert unstructured resume text into machine-readable formats. Common representation techniques include:

- Text-Based Representation**:** The body is represented as a collection of keypoints (joints) connected to form a skeletal structure. .
- Feature-Based Structured Representation**: I**n this approach, resumes are converted into structured profiles by extracting key attributes such as skills, education, experience, certifications, and job roles. These features enable efficient comparison between candidate profiles and job requirements and are commonly used in applicant tracking systems. he human body.
- Semantic Representation**:**
- Advanced systems represent resumes using semantic embeddings that capture contextual meaning rather than exact keywords. This allows the system to understand skill similarity, role equivalence, and experience relevance, improving the quality of candidate matching and ranking.

2. Resume Screening Approaches

(A) Rule-Based vs. AI-Based Methods:

1. **Rule-Based Screening Approach:**
    - Uses predefined rules and keyword filters based on job descriptions.
    - Resumes are accepted or rejected depending on the presence of required terms.

Advantages: Simple to implement, Fast execution

. **Disadvantages**: Poor semantic understanding, High false rejection rate

2. **AI-Based Screening Approach:**
   - Uses machine learning and natural language processing techniques to analyze resume content.
   - Computes relevance scores between resumes and job descriptions using feature extraction and similarity measures.
   - Supports candidate ranking instead of binary filtering.

Advantages:

- Better handling of synonyms and context.
- Scalable for large applicant pools
- More accurate and flexible ranking

Disadvantages:

- Requires preprocessing and model tuning
- Computationally heavier than rule-based systems

(B) Machine Learning-Based vs. NLP-Driven Approaches

1. Machine Learning–Based Models

- Use structured and numerical features derived from resumes
- Common algorithms include Logistic Regression, Naïve Bayes, Support Vector Machines, and Random Forest
- Suitable for binary classification (shortlisted vs. rejected) and relevance scoring.

Advantages:

- Fast inference
- Easy to deploy in production system
- Works well with limited computational resources

Disadvantages:

- Performance depends on feature engineering
- Limited contextual understanding of resume content

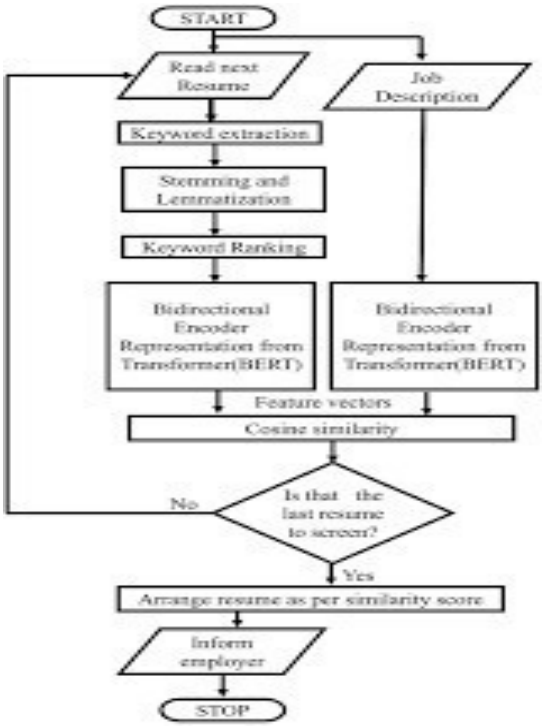


Fig 2.1: Candidate Ranking Flow using Cosine Similarity

2. NLP-Driven Resume Screening Models

- Use text preprocessing, vectorization, and similarity computation
- **Enable context-aware resume–job description matching**

Advantages:

- Better handling of synonyms and varied resume formats.
- Improved relevance matching

Disadvantages:

- Requires careful text preprocessing
- Computationally heavier than basic ML models

3. Key Techniques in AI-Powered Resume Screening

- Text Preprocessing and Normalization**:**

Cleans and standardizes resume text by applying tokenization, stop-word removal, lower-casing, and noise elimination.

- Feature Extraction Using TF-IDF**:**

  Transforms resumes and job descriptions into numerical vectors by assigning weights to terms based on their importance across the document corpus.
- Semantic Similarity Computation**:**

  Generates relevance scores for each candidate and ranks resumes in descending order of suitability. This approach prioritizes the most relevant profiles and supports efficient shortlisting.
- **Multi-Scale Attention Mechanisms:** Focuses on **important body parts while filtering background noise**.

4. Evaluation Metrics for AI-Powered Resume Screening

- **Relevance Score Accuracy:** Measures how well the system ranks relevant resumes higher than irrelevant ones based on job–resume matching.
- Precision**:**

  Indicates the proportion of correctly shortlisted candidates among all selected resumes.

- **Top-K Ranking Accuracy:**

  Evaluates whether the most suitable candidates appear within the top-K ranked results.

5. Challenges in Pose Estimation

- Resume Format Variability**:**

  Resumes differ widely in layout, structure, and file formats, which can affect accurate text extraction and feature representation.
- Skill and Vocabulary Diversity:

  Candidates may describe the same skills or experience using different terminology, making semantic matching challenging
- Bias and Fairness Issues:

  Historical hiring data and biased job descriptions can introduce unintended bias into automated screening and ranking results.

Noise and Irrelevant Information:

Resumes often contain unnecessary content such as graphics, tables, or unrelated details that can interfere with accurate analysis.

2.2 PROJECT RELATED WORK

Existing Research and Developments in AI-Powered Resume Screening

Automated resume screening has gained significant attention due to the increasing volume of job applications and the need for efficient recruitment processes. Research in this domain spans traditional rule-based systems, machine learning approaches, and advanced NLP-driven methods.

1. Traditional Resume Screening Methods

Before the adoption of artificial intelligence, resume screening relied on manual and rule-based techniques, including:

- Rule-Based Applicant Tracking Systems (ATS)**:** Used hard-coded rules related to experience, education, and skills. These systems lacked adaptability and were prone to bias and rigid decision-making.

2. Machine Learning–Based Resume Screening Approaches

The introduction of mUses **self-attention mechanisms** to capture long-range dependencies in images.

machine learning enabled data-driven resume evaluation and classification.

(A) Naïve Bayes and Logistic Regression Models

- Used probabilistic and linear classification techniques to categorize resumes as suitable or unsuitable.
- Introduces **Part Affinity Fields (PAFs)** to establish connections between keypoints.
- **Advantages:** Fast and interpretable
- **Disadvantages:** Requires labeled data.

(B) Support Vector Machines (SVMs)

Advantages: Effective for high-dimensional text data

Disadvantages: Limited scalability for large datasets

(C) Random Forest-Based Screening Systems

- **Strengths:** Handles feature interactions well
- **Limitation:** Less effective in capturing semantic relationships

NLP-Based and Semantic Matching Approaches

Recent research emphasizes semantic understanding rather than simple classification.

(A) TF-IDF and Cosine Similarity Models

Represent resumes and job descriptions as weighted vectors

Strength: Simple, efficient, and interpretable

Limitation: Limited deep contextual understanding

(B) Embedding-Based Resume Matching

Uses word and sentence embeddings to capture semantic similarity between resumes and job descriptions.

Strength: Better handling of synonyms and related skills

Limitation: Higher computational requirements

Ranking Resume Screening & Candidate System Integration

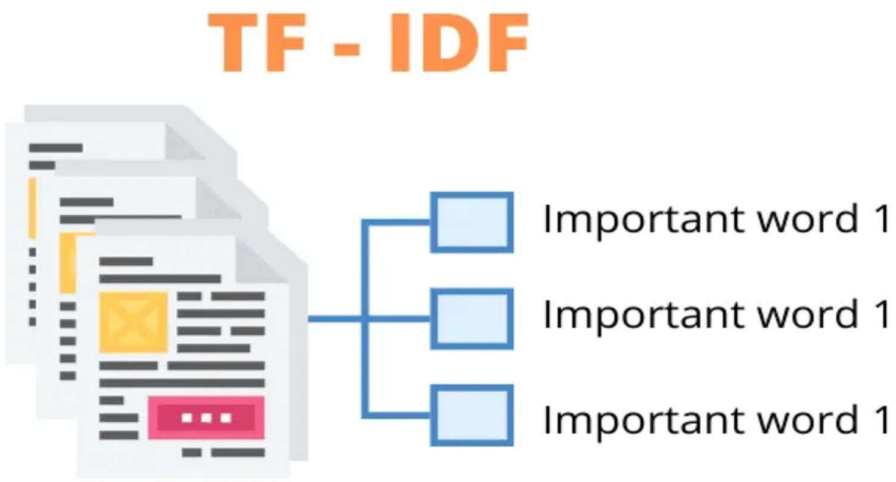


Fig 4: TF-IDF Vectorization and Similarity Process

AI-Powered Resume Screening Platform: I

Integrates natural language processing and machine learning techniques with a full-stack web application to enable automated resume analysis and candidate ranking.

- **Key Strength:**
- Provides relevance-based candidate ranking with high accuracy and efficiency, making it suitable for large-scale recruitment and real-time hiring workflows.

Our Proposed Method: AI-Based Resume Screening and Candidate Ranking Framework (DSCN)

To overcome the limitations of traditional keyword-based and rule-driven recruitment systems, we propose an AI-powered resume screening and candidate ranking framework that emphasizes semantic relevance and scalability.

Key Features of the Proposed System

1. **Resume Parsing Module:**
   - Extracts textual content from resumes in formats such as PDF and DOCX
   - Converts unstructured resumes into analyzable text
2. **Text Preprocessing Module:**
   - Performs tokenization, stop-word removal, normalization, and noise reduction
   - Improves consistency and quality of input data
3. **Feature Extraction Module:**
   - Uses TF-IDF vectorization to identify important skills, experience, and qualifications
4. **Semantic Matching and Scoring Module:**
   - Computes similarity scores between resumes and job descriptions using cosine similarity.
5. **Candidate Ranking Module:**
   - Ranks candidates based on computed relevance scores

Advantages of the Proposed System Over Existing Methods

Improved Matching Accuracy:

Semantic similarity-based scoring ensures relevant candidates are ranked higher.

Reduced Manual Screening Effort:

Automates resume evaluation and ranking, saving recruiter time.

Literature Review:

Title	Author(s)	Year	Algorithm	Disadvantages

Keyword-Based Resume Screening System	Singh et al.	2019	Rule-Based Keyword Matching	Poor semantic understanding, high false rejection rate, unable to handle synonym variations
Machine Learning-Based Resume Classification	Y Zhang et al.	2020	Naïve Bayes, Logistic Regression	Requires labeled datasets, limited adaptability across job domains, ignores contextual relevance
Resume Screening Using TF-IDF and Cosine Similarity	Kumar et al.	2021	TF-IDF Vectorization + Cosine Similarity	Limited deep semantic understanding, sensitive to resume formatting variations
NLP-Based Automated Resume Screening System	Patel et al.	2022	NLP Preprocessing + Similarity Scoring	Performance depends heavily on text preprocessing quality, computationally heavier than rule-based systems
Candidate Ranking Using Semantic Similarity Model	Ahmed et al..	2023	Feature Extraction + Ranking Algorithms r	Potential bias in scoring, requires careful tuning for fair ranking

TABLE - 1

CHAPTER 3: FEASIBILITY STUDY

3.1 TECHNICAL FEASIBILITY

Technical feasibility evaluates whether the proposed AI-powered resume screening and candidate ranking system can be successfully developed and deployed using existing technologies, tools, and infrastructure. It examines the availability of required hardware and software resources and assesses whether the system can be implemented without major technical constraints.

Key Aspects of Technical Feasibility

1. **System Requirements:**
   - The proposed system does not require high-end hardware for deployment, making it suitable for real-world recruitment environments.
   - A full-stack web framework integrates the AI modules with a responsive user interface, Database for storing resumes, extracted features, and ranking results cure database, and RESTful APIs for end-to-end recruitment workflow.

2. **Software Stack:**
   - The system is developed using Python with NLP and machine learning libraries for resume parsing, feature extraction, and relevance-based candidate ranking.
3. **Scalability and Performance:**
   - The system is designed to efficiently process large numbers of resumes with minimal latency.
   - Supports batch processing of resumes
   - Eficient text preprocessing and vectorization reduce processing time
4. **Challenges and Considerations:**
   - Resume Format Variability:
   - Resumes may be submitted in different formats and layouts, requiring robust parsing techniques.
   - **Occlusion Handling:** The system needs to handle cases where certain body joints are not visible.
   - **Adaptability:** The model should work efficiently across different environments (indoor, outdoor, low-light).

Full-Stack Integration:

The AI-powered resume screening system integrates frontend, backend, and database components into a unified platform. Recruiters can upload job descriptions and resumes through the web interface. Backend services handle resume parsing, preprocessing, scoring, and ranking. results are displayed in an intuitive and interpretable format

The architecture supports future enhancements such as advanced NLP models and analytics dashboards

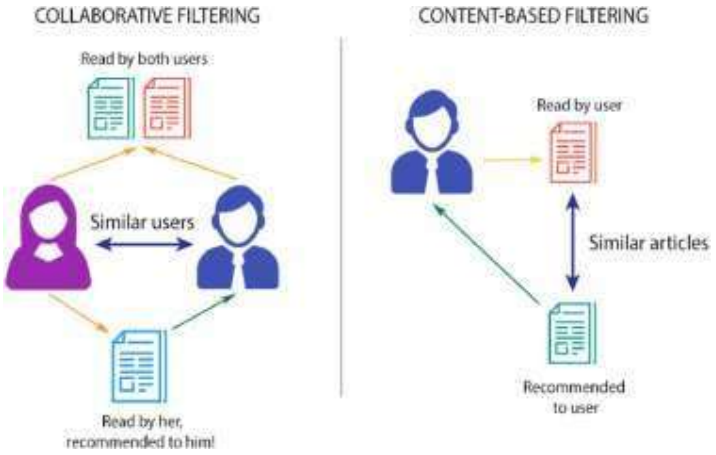


Fig 3.1: Cosine Similarity

AI-powered resume screening typically involves analyzing unstructured resume documents to extract relevant information such as skills, experience, education, and certifications. The system processes resumes using natural language processing (NLP) techniques and converts textual content into numerical representations that can be compared with job descriptions. Models based on TF-IDF vectorization and similarity scoring are commonly used to compute relevance scores, which indicate how well a candidate matches the job requirements. These systems output ranked candidate lists rather than simple accept-or-reject decisions, enabling more informed recruitment outcomes.

One of the key strengths of AI-powered resume screening systems is their efficiency and scalability. Instead of manually reviewing hundreds or thousands of resumes, the system automatically processes large volumes of candidate data in a short time. By using optimized text preprocessing, feature extraction, and similarity computation, the system can perform resume analysis in near real time. This makes it highly suitable for enterprise recruitment platforms and online hiring portals where speed and consistency are critical. The ability to evaluate multiple candidates simultaneously also ensures uniform assessment across the applicant pool.

The integration of AI-based resume screening into recruitment workflows has enabled several practical

applications. In corporate hiring, it helps recruiters quickly shortlist the most relevant candidates for technical and non-technical roles. In campus recruitment, the system can efficiently rank students based on skill alignment and academic background. For online job portals, automated screening improves user experience by matching candidates with suitable job openings. Additionally, candidate ranking systems assist HR teams in reducing bias and improving decision transparency by relying on data-driven evaluation rather than subjective judgment.

Several tools, libraries, and frameworks support the development of AI-powered resume screening systems. NLP libraries are used for text extraction and preprocessing, while machine learning libraries enable feature representation and similarity computation. Full-stack web frameworks integrate these AI modules with user interfaces for resume upload, job description input, and result visualization. The system processes resumes, computes relevance scores against job descriptions, and stores the ranked results in a database for further analysis or recruiter review. This end-to-end pipeline ensures an efficient, scalable, and practical recruitment solution.

Despite its advantages, AI-powered resume screening systems also face certain limitations. One of the primary challenges is **data variability and noise**; resumes differ widely in format, structure, and writing style, which can affect the accuracy of text extraction and feature representation. Additionally, while automated screening significantly improves efficiency, its accuracy may be lower than detailed human evaluation in complex cases where contextual understanding, career gaps, or non-standard experience descriptions are involved.

In conclusion, AI-powered resume screening and candidate ranking systems provide an effective solution for handling large-scale recruitment by automating resume analysis and prioritizing candidates based on relevance. By leveraging NLP and machine learning techniques, these systems enable faster, more consistent, and data-driven hiring decisions. They are particularly suitable for corporate recruitment, campus hiring, and online job portals. However, challenges such as resume variability, contextual ambiguity, and potential bias must be addressed to ensure fair and reliable candidate evaluation.

Graph-Based Modeling in Resume Screening Systems

Graph-based approaches are increasingly explored in AI-powered recruitment systems to model relationships between different resume components such as skills, job roles, experience, and qualifications. In this context, resumes and job requirements can be represented as interconnected entities, where nodes represent skills or attributes and edges represent their relationships or co-occurrence patterns.

This structured representation helps capture dependencies that are difficult to model using traditional flat text representations.

In resume screening tasks, the objective is to assess how well a candidate's profile aligns with a job description. Traditional text-based methods may struggle to capture relationships between skills and experience, especially when information is implicitly stated or distributed across different sections of the resume. Graph-based modeling addresses this limitation by enabling the system to consider relationships between related skills, roles, and experience levels, resulting in more context-aware candidate evaluation.

A key advantage of graph-based techniques in resume screening is their ability to leverage contextual dependencies between resume attributes. For example, when evaluating a candidate's suitability for a role, the system can consider related skills and prior roles together rather than independently. This approach improves robustness when resumes contain incomplete information or unconventional phrasing, leading to more accurate relevance scoring and candidate ranking.

In a typical graph-based resume screening architecture, extracted resume features are organized into a structured representation where relationships between attributes are learned and refined through multiple processing layers. Each layer aggregates information from related attributes, gradually improving the system's understanding of candidate profiles. These models are trained on historical recruitment data to learn patterns that generalize to unseen resumes, ultimately enhancing ranking accuracy and decision reliability.

One of the most significant advantages of an AI-powered resume screening and candidate ranking system is its ability to **process and evaluate large volumes of resumes simultaneously** with consistent accuracy. By representing resumes and job descriptions as high-dimensional feature vectors using techniques such as TF-IDF, semantic embeddings, or transformer-based representations, the system can rank hundreds or thousands of candidates in seconds. This capability is particularly valuable in real-world recruitment scenarios where manual screening is slow, biased, and error-prone, especially in domains like IT hiring, campus recruitment, and large-scale corporate onboarding.

The candidate ranking mechanism leverages machine learning models to **capture semantic relevance rather than keyword matching**, ensuring that candidates with equivalent skills expressed differently are not unfairly rejected. By modeling the relationship between resumes, job requirements, and historical hiring data, the system can prioritize candidates whose skill sets, experience levels, and domain expertise

align most closely with the job role. When extended across multiple job profiles, the system can dynamically adapt ranking criteria, enabling scalable and role-specific screening without human intervention.

In summary, an AI-powered resume screening and candidate ranking system provides a robust, scalable, and objective approach to modern recruitment. By modeling resumes and job descriptions through intelligent feature representations and ranking algorithms, the system enhances hiring accuracy, reduces bias, and improves operational efficiency. Whether applied to small organizations or enterprise-level recruitment pipelines, this approach offers a practical and future-ready solution for intelligent talent acquisition.

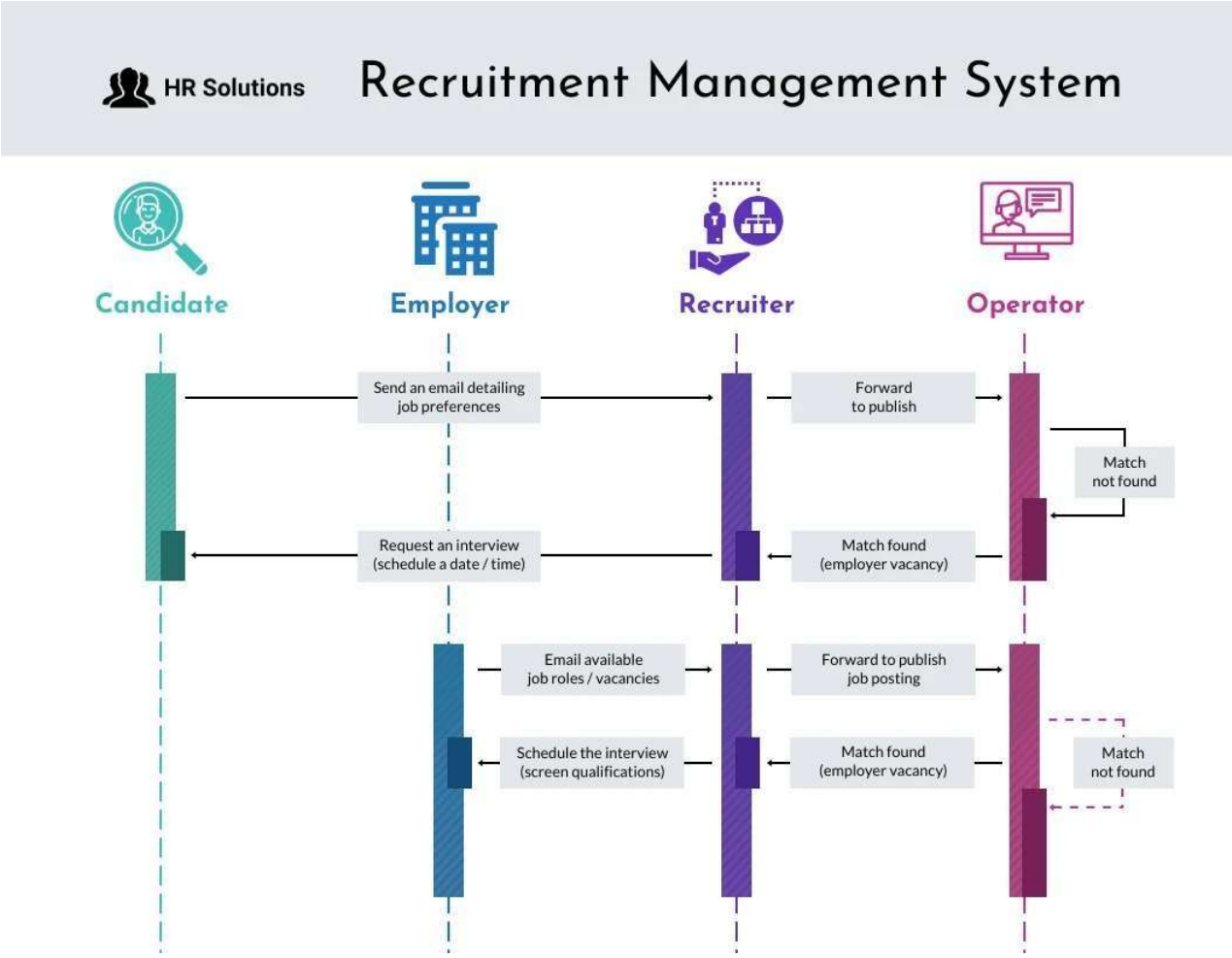


Fig 3.2: Sequence Diagram

3.2 OPERATIONAL FEASIBILITY

Operational feasibility evaluates whether the proposed AI-powered resume screening and candidate ranking system can be effectively deployed in real-world recruitment environments

Key Aspects of Operational Feasibility

3.2.1 User Requirements:

3.2.1.1 The system must be easy to use and require minimal technical knowledge from recruiters.

3.2.1.2 The system should be adaptable to various recruitment domains including IT, healthcare, finance, and academic hiring

3.2.2 Ease of Integration:

3.2.2.1 TAPI-based architecture enables seamless integration with web portals and enterprise hiring applications.

3.2.2.2 Supports cloud or local deployment depending on organizational requirements.

3.2.2.3 The system can be integrated with existing recruitment platforms and Applicant Tracking Systems (ATS)

3.2.3 Training and Usability:

3.2.3.1 Minimal training is required as the resume screening and ranking process is automated.

3.2.3.2 A simple and intuitive web-based dashboard allows recruiters to upload job descriptions, manage resumes, and view ranked candidates.

3.2.3.3 Clear visualization of candidate scores and rankings improves decision-making

3.2.4 Real-World Use Cases:

3.2.4.1 **Corporate Recruitment:** Automates resume shortlisting and ranking for large-scale hiring.

3.2.4.2 **HR Consultancies:** Improves efficiency in screening candidates for multiple clients

3.2.4.3 **Campus Recruitment:** Assists institutions in filtering and ranking student resumes.

3.2.4.4 **Startup Hiring:** Enables fast and unbiased candidate selection with limited HR resources.

3.2.5 Challenges in Implementation:

3.2.5.1 Requires an **initial setup** with a **camera and computing hardware**.

3.2.5.2 Needs **calibration** for different environments (e.g., lighting conditions, backgrounds).

Conclusion on Operational Feasibility

The AI-powered resume screening and candidate ranking system is operationally feasible as it can be easily integrated into existing recruitment workflows, requires minimal user training, and efficiently handles large volumes of resumes with high accuracy and reliability.

3.3 ECONOMIC FEASIBILITY

Economic feasibility evaluates whether the proposed AI-powered resume screening and candidate ranking system is cost-effective and capable of delivering a high return on investment (ROI) for organizations.

Cost Analysis

3.3.1 Development Costs:

3.3.1.1 Hardware Costs:

3.3.1.1.1 Development Laptop / Server (8–16 GB RAM, SSD) – ₹40,000–₹80,000

3.3.1.1.2 Optional Cloud Compute (CPU-based, no high-end GPU required) – ₹1,000–₹5,000/month

3.3.1.2 Software Costs:

3.3.2 Deployment Costs:

3.3.2.1 Cloud deployment using platforms such as AWS, Google Cloud, or Azure (basic compute instances).

3.3.2.2 On-premises deployment possible for organizations with existing infrastructure. No specialized hardware required, reducing overall deployment expenses.

3.3.3 Operational Costs:

3.3.3.1 Minimal electricity consumption compared to GPU-intensive systems.

3.3.3.2 Periodic system maintenance and model updates

3.3.3.3 Optional cloud storage and bandwidth charges for large-scale resume data handling.

3.3.4 Revenue and ROI Potential:

3.3.4.1 Can be deployed as an internal recruitment automation tool to reduce HR operational costs.

3.3.4.2 Subscription-based SaaS (Software-as-a-Service) model for companies and recruitment agencies.

3.3.4.3 API-based licensing for integration with Applicant Tracking Systems (ATS) and HR platforms.

3.3.4.4 Reduces manual screening time, leading to significant cost savings and productivity gains.

3.3.5 Cost-Benefit Analysis:

3.3.5.1 Low initial development and deployment cost due to open-source tools and lightweight models.

3.3.5.2 Minimal recurring operational expenses.

3.3.5.3 High efficiency and scalability compared to traditional manual and rule-based screening systems.

Faster hiring cycles and improved candidate matching increase overall organizational ROI.

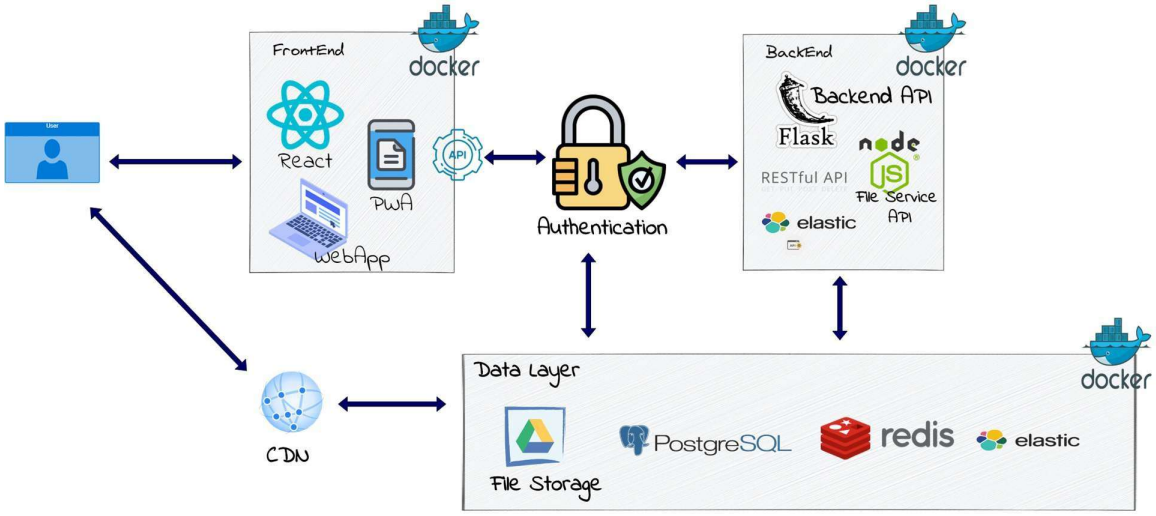


Fig 3.3: Deployment Diagram

CHAPTER 4: SOFTWARE REQUIREMENT SPECIFICATIONS

4.1 FUNCTIONAL REQUIREMENTS

Functional requirements define the core functionalities of the system, ensuring that the AI-powered resume screening and candidate ranking system performs as expected.

1. Automated Resume Screening

- The system must accept resumes in common formats such as PDF and DOCX.
- It should automatically extract relevant information including skills, experience, education, and certifications.

The system must handle both single and bulk resume uploads.

2. Job Description Processing

- The system should allow recruiters to upload or input job descriptions.
- It must analyze job requirements and convert them into structured feature representations.
- The system should support multiple job roles and categories simultaneously.

3. Candidate Ranking

- The system must compute relevance scores between resumes and job descriptions
- It should rank candidates based on overall suitability and skill relevance..
- The ranking process must be consistent and free from manual bias.

4. Accuracy and Matching Performance

- The system should achieve high matching accuracy for relevant candidates.
- It must correctly identify skill and experience alignment even when different terminology is used.
- The ranking results should be stable and reproducible for the same inputs.

5. Resume Data Storage and Export

- The system must store processed resume data securely in a database.
- Users should be able to export screening results in formats such as CSV or Excel.
- The system should allow downloading ranked candidate lists for offline review.

6. User Interface and API Support

- Provide a web-based dashboard for uploading resumes and job descriptions.
- Display ranked candidates with relevance scores in a clear and interactive interface.

Offer REST API support for integration with Applicant Tracking Systems (ATS)

4.2 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define the quality attributes of the system, ensuring that the AI-powered resume screening and candidate ranking system operates efficiently, reliably, and securely in real-world environments.

4.2.1 PERFORMANCE REQUIREMENTS

- The system must process and analyze resumes with low latency to support near real-time screening.
- Resume parsing and feature extraction should complete within **2–5 seconds per resume** on standard systems.
- The system should support **bulk resume processing** (at least 100–500 resumes) without significant performance degradation. Candidate ranking results should be generated immediately after resume analysis.
- The system must handle concurrent users efficiently in a web-based environment. Backend services should be optimized to ensure smooth interaction between frontend.

4.2.2 HARDWARE REQUIREMENTS

The hardware requirements depend on whether the model is being used for **training** or **deployment**.

1. Hardware for Model Training (High-Performance System)

- **Processor:** Intel Core i5/i7 or AMD Ryzen 5/7 (or higher).
- **GPU (Optional):** NVIDIA GTX 1650 / RTX 2060 or higher (for faster experimentation)
- **RAM:** Minimum 8GB (16GB recommended
- **Storage:** 256GB–512GB SSD.
- **Operating System:** Windows / Linux

2. Hardware for Deployment (Web Application & Server)

- **Processor:** Intel i3/i5 or equivalent cloud CPU
- **GPU:**
- Not mandatory (CPU-based inference sufficient)
- **RAM**: Minimum 4GB (8GB recommended for scalability) t
- **Storage: 100GB SSD or cloud storage**
- **Network: Stable internet connection for cloud-based deployment**

4.2.3 SOFTWARE REQUIREMENTS

The system relies on various software tools for development, model training, deployment, and visualization of the AI-powered resume screening and candidate ranking system.

1. Development Environment

- **Operating System:**
  - Windows 10/11, Ubuntu 20.04+, macOS.
- **Programming Languages:**
  - Python 3.8+ (for resume parsing, NLP processing, machine learning models, and backend services)
  - **JavaScript** (for frontend development and client-side interactions)

2. Machine Learning and NLP Frameworks

- **Scikit-learn** (for TF-IDF vectorization and machine learning models)
- **NLTK / SpaCy** (for natural language processing and text analysis)
- **Sentence Transformers (optional)** for semantic similarity computation.

Web Development and Deployment Frameworks:

Flask / FastAPI (for backend API development)

REST API for communication between frontend and backend services

3. Database and Storage:

- **SQLite / PostgreSQL** (for storing resume data and ranking results

- **Flask / FastAPI** (for API deployment).

4.2.4 INTRODUCTION TO PYTHON / WEB TECHNOLOGIES

Introduction to Python

Python is a **high-level, interpreted programming language** that has become the industry standard for **machine learning and deep learning** applications.

Key Features of Python:

- **Simple and Readable Syntax:** Easier to learn and use.
- **Extensive Library Support:** Includes libraries like **NumPy, Pandas, OpenCV, and TensorFlow**.
- **Cross-Platform Compatibility:** Runs on Windows, macOS, and Linux.
- **Support for Deep Learning Frameworks:** Works with **PyTorch, TensorFlow, and Keras**.
- **Multi-Paradigm:** Supports **OOP (Object-Oriented Programming), functional programming, and procedural programming**.

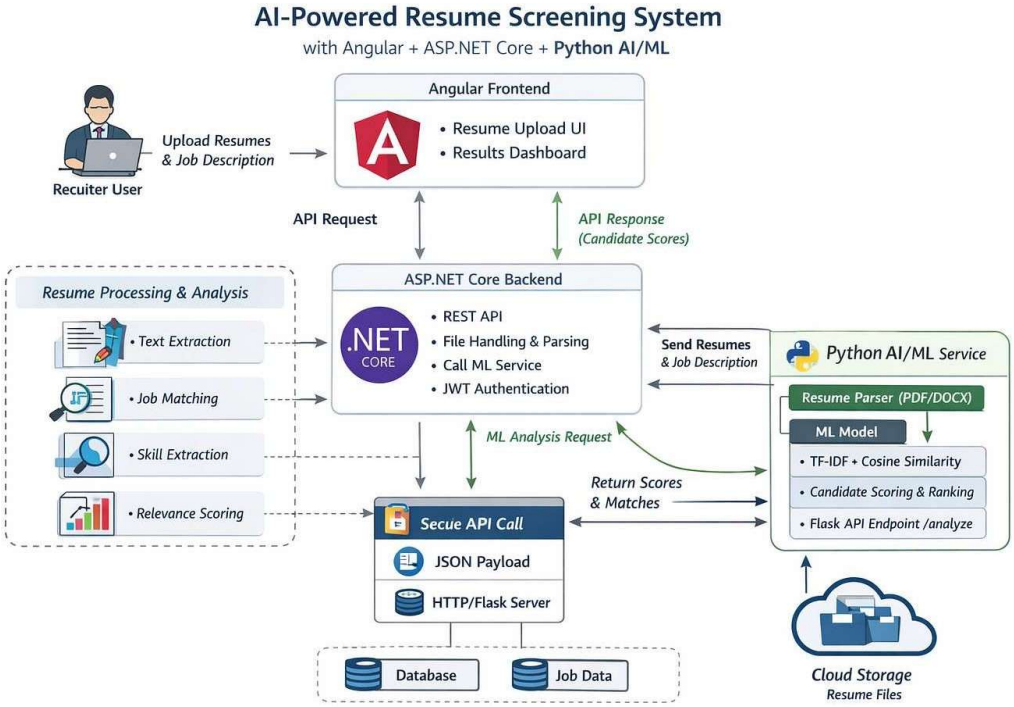


Fig 4.1: System Architecture Diagram (Detailed)

Why Python for Resume Screening and Candidate Ranking?

- **Natural Language Processing:** Efficiently processes unstructured resume text and job descriptions.
- **Machine Learning Integration:** Supports feature extraction, similarity scoring, and ranking algorithms.
- **Backend Development:** Easily integrates with Flask/FastAPI for full-stack applications.
- **Scalability:** Suitable for both local and cloud-based deployments.

Introduction to Java

Java is a widely used, object-oriented programming language known for its platform independence, scalability, and strong security features. It is commonly employed in enterprise-level applications, web services, and large-scale backend systems.

Key Features of Java:

- **Write Once, Run Anywhere (WORA):** Java applications can run on any OS with a **Java Virtual Machine (JVM)**.
- **Strong Memory Management:** Uses **Garbage Collection (GC)** for automatic memory handling.
- **Multi-Threading Capabilities:** Allows concurrent execution of multiple processes.
- **Security:** Provides **robust security features** for web applications.

Java in AI-Powered Resume Screening

While Python is primarily used for machine learning and natural language processing, Java plays an important role in **enterprise-level integration and backend services** for resume screening systems. Java can be used to build scalable recruitment platforms, RESTful APIs, and secure web applications that integrate with AI models developed in Python.

Java vs. Python for Resume Screening and Candidate Ranking

Feature	Python	Java
Ease of Use	Simple & readable	+ More complex
NLP & ML Support	Strong (Scikit-learn, SpaCy, Transformers)	+ Limited native ML libraries

Development Speed	Faster prototyping	Slower development
Web & Enterprise Use	Backend APIs (Flask/FastAPI)	Strong enterprise support

Java Use Cases in AI-Powered Resume Screening

- **Enterprise Recruitment Applications**
  - Java is widely used in enterprise software development, making it suitable for building large-scale recruitment and human resource management systems. Java-based applications can integrate AI-powered resume screening services to automate candidate shortlisting and ranking.

- **Backend and API Integration**
  - Java can be used to develop secure and scalable backend services that communicate with AI models through RESTful APIs. This allows seamless integration of resume screening functionality into existing Applicant Tracking Systems (ATS) and recruitment platforms.

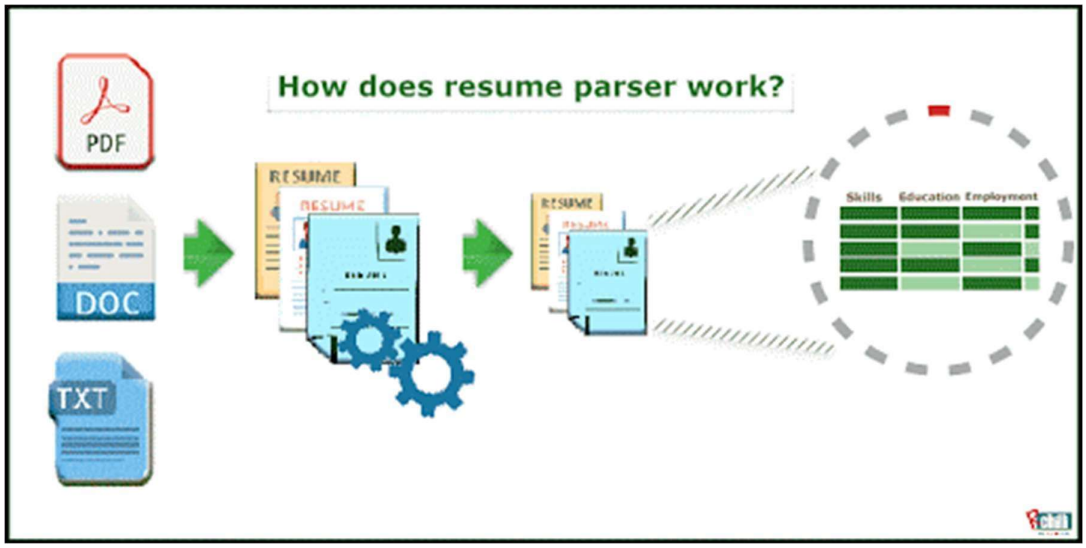


Fig 4.2: Resume Parsing and Feature Extraction Process

CHAPTER 5: SYSTEM MODELING

5.1 REQUIREMENTS MODELING

Requirements modeling defines the structure, behavior, and interactions within the **AI-Powered Resume Screening and Candidate Ranking System**. This phase focuses on representing how different system components interact with each other to achieve automated resume analysis, candidate ranking, and recruiter interaction.

5.1.1 CLASS DIAGRAMS

A **Class Diagram** provides a static view of the system structure by illustrating the classes involved in the resume screening system along with their attributes, methods, and relationships.

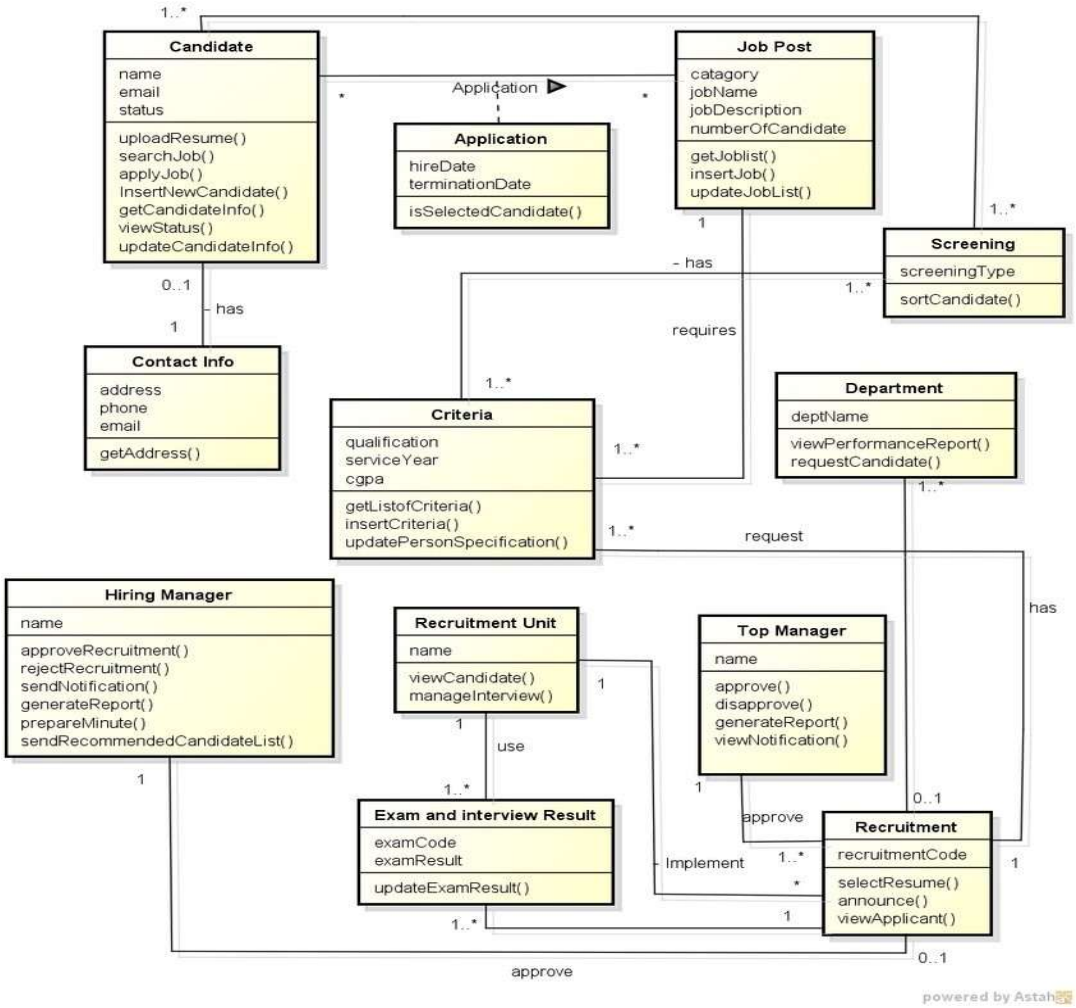


Fig 5.1: Class Diagram

Class Relationships

- **Resume Parser** uses **NLP Processor** to extract and preprocess text from uploaded resumes and job descriptions.
- **Feature Extractor** sends processed text to the **Machine Learning Model** for relevance scoring and candidate evaluation
- **Ranking Engine** generates ranked candidate lists.

5.1.2 USE CASE DIAGRAMS

A Use Case Diagram describes how different users and system components interact with the AI-powered resume screening and candidate ranking system.

Actors:

- **Recruiter / HR User:** Uploads resumes and job descriptions, views ranked candidates.
- **Resume Upload Module:** Accepts and validates resume files.
- **AI Screening Engine:** Processes resumes, extracts features, and evaluates candidate relevance keypoints.
- **Database System:** Stores resumes, job descriptions, scores, ranking

retrieval.

Primary Use Cases

1. **Recruiter Uploads Resume(s)** → System parses and preprocesses the documents
2. **Recruiter Submits Job Description** → System analyzes job requirements.
3. **AI Engine Performs Resume Screening** → Extracts skills and computes relevance scores.).
4. **Candidate Ranking Generation** → System ranks candidates based on suitability.

This ensures a clear workflow for different functionalities.

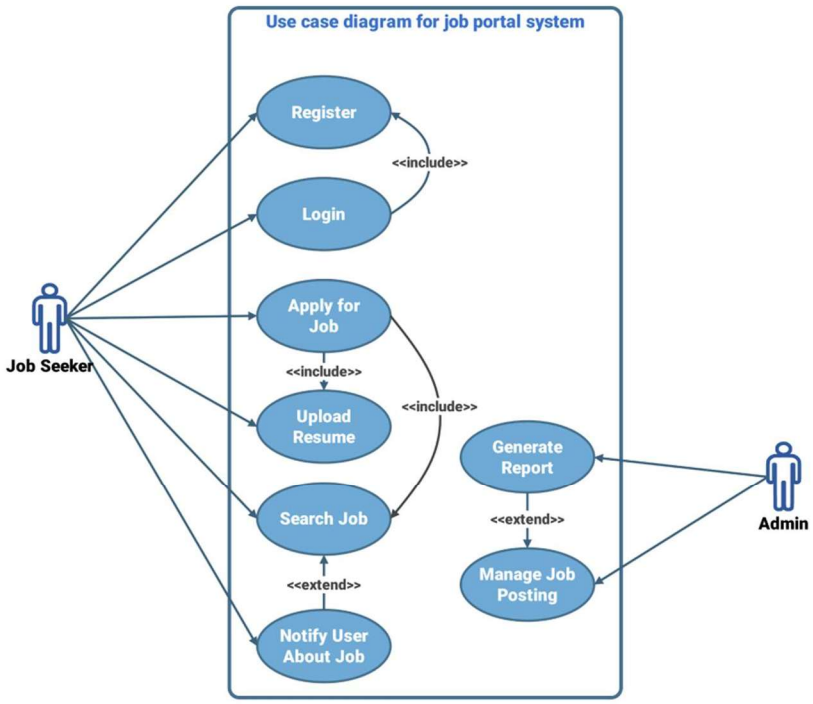


Fig 5.1.2: Use Case Diagram

5.1.3 SEQUENCE DIAGRAMS

A **Sequence Diagram** illustrates the chronological flow of interactions between different system components involved in the AI-powered resume screening and candidate ranking process.

Step-by-Step Workflow

1. **Recruiter initiates resume screening** → The system activates the Resume Upload and Processing
2. **ResumeParser** extracts and preprocesses text from uploaded resumes and job descriptions.
3. **FeatureExtractor** converts processed text into numerical representations using NLP techniques.
4. **MachineLearningModel** analyzes features and computes relevance scores for each candidate.
5. **RankingEngine** sorts candidates based on suitability scores.
6. **DatabaseManager** stores screening results and ranking information.

This structure ensures **efficient message passing for real-time execution**.

5.1.4 COLLABORATION DIAGRAMS

A **Collaboration Diagram** represents **how components interact** dynamically.

Major

1. **Recruiter Interface** sends resume files and job descriptions to the **ResumeParser** module.
2. **ResumeParser** extracts and preprocesses textual information and forwards it to the **FeatureExtractor**.
3. **MachineLearningModel** computes relevance scores and sends them to the **RankingEngine**..
4. **RankingEngine** organizes candidates based on suitability and returns the ranked results.
5. **DatabaseManager stores processed data and ranking outputs for retrieval.**

This modular interaction enables **scalability and efficient debugging**.

5.1.5 ACTIVITY DIAGRAMS

An **Activity Diagram** models **step-by-step execution**.

Resume Screening and Ranking Workflow

1. **Start** → System Initialization.
2. **User Authentication check:**

If authentication fails → Exit process.

3. **Resume Upload:**

Recruiter uploads single or multiple resumes.

System validates file format and content.

4. **Job Description Input:**
   - Recruiter submits job description.
   - System preprocesses and validates job requirements.
5. **Resume Processing:**

Extract text from resumes.

Perform text cleaning and preprocessing.

6. **AI-Based Resume Screening:**

   Apply NLP-based feature extraction.

   Compute semantic similarity between resumes and job description.

7. **Candidate Ranking:**

   Rank candidates based on suitability scores.

8. **Display Results on the UI.:**

   Ranked candidate list is shown to the recruiter

9. **User Decision:**

   - Export results?
   - Yes → Store data (CSV/Excel).
   - No → Continue screening.

Repeat Process for another job role or **Exit System**.

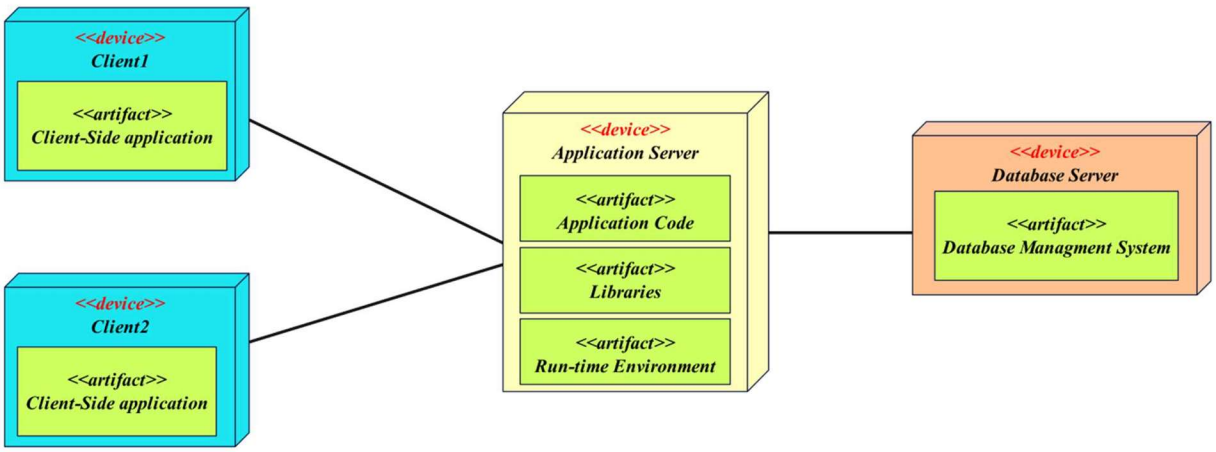


Fig 5.1.5: Activity Diagram

5.1.6 DEPLOYMENT DIAGRAMS

A Deployment Diagram illustrates the physical and logical deployment of software components across hardware nodes, ensuring scalability, performance, and security for the AI-powered Resume Screening and Candidate Ranking System.

1. Hardware Layer

Client Devices (HR/Admin/Recruiter)

- Desktop / Laptop / Mobile Browser
- Used to upload resumes, view rankings, and manage candidates

AI Processing Server (GPU/High-Performance CPU)

Executes NLP and machine learning models

2. Software Layer

- Frontend Layer**:** React / HTML / JavaScript
- Backend Layer**:** Flask / Django / Node.js
- Database Layer**:** MySQL / PostgreSQL / MongoDB.

This ensures **optimal system performance for real-time applications**.

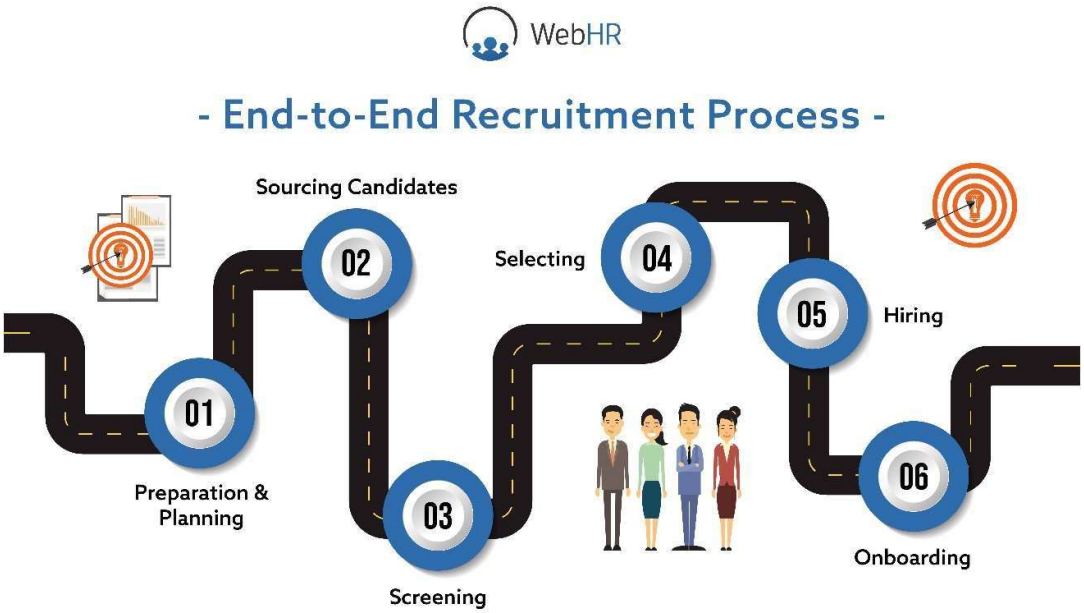


Fig 5.1.6: Deployment Diagram

5.1.6 MODULES DESCRIPTION

1. Input & Data Ingestion Module

- Accepts resumes in **PDF, DOCX, and TXT** formats.
- Allows HR to upload **job descriptions** and candidate profiles.
- Validates file structure and removes corrupted or unsupported files.

2. Preprocessing Module

- Converts resumes into machine-readable text using parsing techniques.
- Cleans text by removing noise such as symbols, stop words, and formatting artifacts.
- Performs **tokenization, lemmatization**

3. Resume Parsing Module

- Extracts structured information such as **skills, education, experience, certifications, and keywords**.
- Identifies section headers and context-aware entities using NLP techniques.
- Transforms unstructured resume data into a structured format.

4. Feature ectraction&Embedding Module

- Converts textual data into numerical vectors using **TF-IDF, word embeddings, or transformer-based encoders**..
- Extracts semantic representations from resumes and job descriptions.
- Enables accurate similarity comparison

5. Candidate Ranking & Scoring Module

- Computes relevance scores between resumes and job descriptions.
- Ranks candidates based on skill match, experience relevance, and contextual similarity.
- Applies machine learning classifiers or similarity-based ranking algorithms.

6. Post-Processing & Decision Module

Applies thresholding to filter unsuitable candidates.

Handles missing or incomplete resume information gracefully.

Reduces bias by normalizing scores across candidate pools.

7. Visualization & HR Dashboard Module:

Displays ranked candidate lists with confidence scores.

Provides filters for skills, experience, education, and role fit.

Enables real-time interaction for HR decision-making.

5.2 DESIGN MODELING

Design modeling defines a **clear, modular, and scalable architecture** for the AI-powered Resume Screening and Candidate Ranking System, ensuring efficient data flow between the frontend, backend, and AI components.

5.2.1 SYSTEM ARCHITECTURE

The system follows a **dual-stream AI-driven architecture** combining **resume understanding** and **job requirement analysis**, integrated through a centralized ranking mechanism.

1. Input Stream

- Accepts resumes in **PDF/DOCX/TXT** formats from candidates or HR users.
- Collects **job descriptions, skill requirements, and role criteria**.
- Performs initial validation and forwards data for preprocessing.

2. Resume Analysis Stream

- Applies **NLP-based preprocessing** (cleaning, tokenization, normalization).
- Uses **TF-IDF / word embeddings / transformer encoders** for feature extraction.
- Converts unstructured resume content into structured semantic representations.

3. Job Description Analysis Stream

- Processes job descriptions using the same NLP pipeline as resumes.
- Extracts required skills, experience levels, and role-specific keywords
- Generates comparable feature vectors for accurate matching.

4. Feature Fusion & Matching Module

- Aligns resume vectors with job description vectors
- Uses **semantic similarity scoring and weighted feature matching**.
- Handles variations in terminology and contextual meaning.

5. Candidate Ranking & Scoring Module

- Computes relevance scores using **machine learning classifiers or similarity models**.
- Ranks candidates based on skill match, experience relevance, and contextual fit.

- Normalizes scores to ensure fair comparison across large candidate pools.

6. Output & Decision Module

- Produces a **ranked list of candidates** with confidence scores.
- Flags top candidates and filters out low-relevance profiles.
- Supports HR decision-making with explainable ranking indicators.

7. Visualization & User Interface Module

Displays results through a **full-stack web dashboard**.

Enables filtering, sorting, and shortlisting of candidates.

Provides real-time interaction for recruiters and administrators.

8. Storage & Export Module

Stores resumes, extracted features, scores, and rankings in databases.

Supports export in **CSV, Excel, or PDF** formats.

Integrates with **cloud storage** for scalability and backup.

CHAPTER 6: SYSTEM IMPLEMENTATION

- System implementation represents the phase where the designed architecture of the **AI-Powered Resume Screening and Candidate Ranking System** is transformed into a fully operational full-stack web application.
- This phase begins with configuring the **development environment**, including frontend frameworks, backend servers, databases, and machine learning libraries required for natural language processing and model inference.
- The **frontend layer** is implemented using modern web technologies to provide an intuitive interface for resume upload, job description input, candidate ranking, and analytics visualization. Responsive design principles are followed to ensure accessibility across devices.
- The **backend layer** is developed to manage application logic, user authentication, resume processing workflows, and secure communication between system components through RESTful APIs.
- The **AI processing layer** is implemented using machine learning and NLP techniques to parse resumes, extract features, compute semantic similarity, and generate relevance scores for candidate ranking
- **Debugging and optimization** are performed to resolve parsing errors, ranking inconsistencies, and performance bottlenecks, ensuring reliable and efficient system behavior.
- The system components are then **integrated and deployed** in the target environment, enabling secure resume storage, scalable processing, and seamless user interaction.
- Detailed **documentation** is prepared to describe implementation logic, system configuration, and deployment steps, supporting future maintenance and enhancements.
- Following deployment, **user guidance and support** are provided to help HR personnel and administrators effectively utilize the resume screening and candidate ranking features.
- Successful implementation results in a **robust, scalable, and intelligent recruitment system** capable of automating resume screening and improving hiring efficiency.

6.1 DEVELOPMENT ENVIRONMENT AND TOOLS

The development environment comprises the **hardware, software, and tools** required for designing, developing, training, testing, and deploying the **AI-Powered Resume Screening and Candidate Ranking System**.

Hardware Requirements

The hardware requirements vary based on **development, model training, and deployment** stages.

1. Development & Training Environment:

- **Processor:** Intel Core i7 / i9 or AMD Ryzen 7 / 9.
- **GPU:** NVIDIA GTX 1660 / RTX 2060 or higher (optional but recommended for faster model training)
- **RAM:** Minimum 16GB (32GB recommended for NLP model training)
- **Storage:** 1TB SSD for storing large datasets.
- **Power Supply:** 750W PSU for stable GPU operations.

2. Deployment Environment (Edge Devices):

- **Processor:** ARM Cortex-A72 (for Raspberry Pi), Intel i5+ (for standard devices).
- **GPU:** NVIDIA Jetson Xavier NX, Google Coral TPU.
- **RAM:** Minimum 8GB for smooth inference.
- Storage**:** 512GB / 1TB SSD for datasets, resumes, and trained models

Software Tools

The software stack integrates **web technologies, backend services, databases, and AI frameworks**.

1. Operating System:

- Windows 10 / 11,Ubuntu 20.04+,macOS (for development)

2. Programming Languages:

- **Python 3.8+** – Primary language for AI, NLP, and backend logic
- **JavaScript** – Frontend development and API interaction

3. AI & Machine Learning Frameworks:

- **Scikit-learn** – Classification and ranking models
- **TensorFlow / PyTorch** – Advanced NLP and deep learning models.
- NLP Libraries: NLTK / spaCy.

4. Feature Extraction & Ranking Tools:

- **TF-IDF Vectorizer** – Text vectorization
- **Word Embeddings / Sentence Embeddings** – Semantic similarity

- Machine Learning Models:Machine Learning Models, Similarity-based ranking algorithms

5. Backend & API Development:

- **Flask / Django / FastAPI** – Backend services and REST APIs
- **JWT / OAuth** – Authentication and authorization
- **PDF & DOCX Parsers:** PyPDF2, python-docx

6. Frontend Development Tools

React.js / HTML / CSS / Bootstrap

Chart.js / Recharts – Visualization of candidate rankings

7. Database & Storage:

MySQL / PostgreSQL – Structured candidate data

MongoDB – Unstructured resume data

Cloud Storage – Resume file storage and backups

8. Deployment & DevOps Tools:

Docker – Containerization

Git / GitHub – Version control

Cloud Platforms: AWS / Azure / Google Cloud

These tools ensure **efficient model training, optimization, and real-time execution**.

Source Code:

```
import os
import uuid
import csv
import re
from io import StringIO
from datetime import datetime
from flask import Flask, render_template, redirect, session, flash, request, make_response
from PyPDF2 import PdfReader
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from firebase_config import db, firebase_auth

app = Flask(
    __name__,
    template_folder=os.path.join("..", "frontend", "templates"),
    static_folder=os.path.join("..", "frontend", "static")
)

app.secret_key = "recruitai_secure_key_v2"

# ============================
# HELPERS
# ============================

def parse_pdf(f):
    try:
        return "".join([p.extract_text() for p in PdfReader(f).pages])
    except:
        return ""
```

```python
def get_ai_score(t1, t2):
    if not t1 or not t2:
        return 0
    try:
        return round(cosine_similarity(
            TfidfVectorizer().fit_transform([t1, t2])
        )[0][1] * 100, 2)
    except:
        return 0

def get_details(txt):
    email = re.search(r'[\w\.-]+@[\w\.-]+', txt)
    name = txt.split("\n")[0][:30]
    return name, email.group(0) if email else "Unknown"

def login_required():
    return 'user' in session

# ============================
# AUTH ROUTES
# ============================

@app.route('/')
def home():
    if login_required():
        return redirect('/dashboard')
    return render_template("login.html")
```

```python
@app.route('/signup', methods=['POST'])
def signup():
    try:
        firebase_auth.create_user(
            email=request.form['email'],
            password=request.form['password']
        )
        flash("Signup successful. Please login.", "success")
    except Exception as e:
        flash(str(e), "error")
    return redirect('/')

@app.route('/delete_all_data', methods=['POST'])
def delete_all_data():

    if 'user' not in session:
        return redirect('/')

    uid = session['user']['id']

    batch = db.batch()

    # candidates
    for doc in db.collection("candidates").where("user_id","==",uid).stream():
        batch.delete(doc.reference)

    # jobs
    for doc in db.collection("job_roles").where("user_id","==",uid).stream():
        batch.delete(doc.reference)
```

```python
def delete_all_data():
    batch.commit()

    flash("All data deleted successfully.", "success")

    return redirect('/admin')

@app.route('/ranking')
def ranking():

    if 'user' not in session:
        return redirect('/')

    uid = session['user']['id']

    docs = db.collection("candidates").where("user_id", "==", uid).stream()

    grouped = {}

    for d in docs:
        c = d.to_dict()
        c["id"] = d.id

        role = c.get("matched_role", "Unknown Role")

        if role not in grouped:
            grouped[role] = []

        grouped[role].append(c)
```

```python
def ranking():
    # sort by score
    for role in grouped:
        grouped[role] = sorted(grouped[role], key=lambda x: x["score"], reverse=True)

    return render_template("ranking.html", grouped=grouped)

@app.route('/login', methods=['POST'])
def login():
    try:
        user = firebase_auth.get_user_by_email(request.form['email'])
        session['user'] = {'id': user.uid, 'email': user.email}
        return redirect('/dashboard')
    except:
        flash("Invalid credentials", "error")
        return redirect('/')

@app.route('/logout')
def logout():
    session.clear()
    return redirect('/')
@app.route('/google-login', methods=['POST'])
def google_login():
    try:
        data = request.get_json()
        token = data.get("token")

        # Verify Firebase token
        decoded_token = firebase_auth.verify_id_token(token)
```

```python
def google_login():
        # Create session
        session['user'] = {
            "id": decoded_token["uid"],
            "email": decoded_token.get("email")
        }

        return {"status": "success"}

    except Exception as e:
        return {"status": "error", "message": str(e)}, 401
# ============================
# DASHBOARD
# ============================

@app.route('/dashboard')
def dashboard():
    if not login_required():
        return redirect('/')

    uid = session['user']['id']

    jobs = list(db.collection("job_roles").where("user_id", "==", uid).stream())
    cands = list(db.collection("candidates").where("user_id", "==", uid).stream())

    stats = {
        "jobs": len(jobs),
        "cands": len(cands),
        "short": len([c for c in cands if c.to_dict().get("status") == "Shortlisted"]),
```

```python
def dashboard():
        "avg": round(sum([c.to_dict().get("score", 0) for c in cands]) / len(cands), 1) if cands el
    }

    recent = sorted(
        [c.to_dict() | {"id": c.id} for c in cands],
        key=lambda x: x.get("created_at", datetime.utcnow()),
        reverse=True
    )[:5]

    return render_template("dashboard.html", stats=stats, recent=recent)

# ============================
# JOBS
# ============================

@app.route('/job_roles')
def job_roles():
    if not login_required():
        return redirect('/')

    uid = session['user']['id']
    jobs = db.collection("job_roles").where("user_id", "==", uid).stream()
    jobs = [j.to_dict() | {"id": j.id} for j in jobs]

    return render_template("jobs.html", jobs=jobs)

@app.route('/add_job', methods=['POST'])
def add_job():
```

```python
def add_job():
    uid = session['user']['id']

    for f in request.files.getlist('files'):
        text = parse_pdf(f)
        if text:
            db.collection("job_roles").add({
                "user_id": uid,
                "title": f.filename,
                "description": text,
                "created_at": datetime.utcnow()
            })
    return redirect('/job_roles')
@app.route('/admin')
def admin():

    if 'user' not in session:
        return redirect('/')

    uid = session['user']['id']

    jobs = list(db.collection("job_roles").where("user_id","==",uid).stream())
    cands = list(db.collection("candidates").where("user_id","==",uid).stream())

    stats = {
        "jobs": len(jobs),
        "cands": len(cands)
    }
```

```python
def admin():
    return render_template("admin.html", stats=stats)


@app.route('/delete_job', methods=['POST'])
def delete_job():
    db.collection("job_roles").document(request.form['id']).delete()
    return redirect('/job_roles')

# ============================
# AI SCREENING
# ============================

@app.route('/ai_screening')
def screening():
    return render_template("screen.html")

@app.route('/process', methods=['POST'])
def process():
    uid = session['user']['id']
    jobs = db.collection("job_roles").where("user_id", "==", uid).stream()
    jobs = [j.to_dict() | {"id": j.id} for j in jobs]

    for f in request.files.getlist('resumes'):
        txt = parse_pdf(f)
        if not txt:
            continue

        best_job = None
```

```
def process():
        best_score = -1

        for j in jobs:
            score = get_ai_score(txt, j['description'])
            if score > best_score:
                best_score = score
                best_job = j

        if best_job:
            name, email = get_details(txt)
            status = "Shortlisted" if best_score >= 75 else "On Hold" if best_score >= 50 else "Rejec

            db.collection("candidates").add({
                "user_id": uid,
                "name": name,
                "email": email,
                "job_role_id": best_job['id'],
                "matched_role": best_job['title'],
                "score": best_score,
                "status": status,
                "resume_text": txt[:5000],
                "created_at": datetime.utcnow()
            })

    return redirect('/candidates')

# ============================
# CANDIDATES
```

```
@app.route('/candidates')
def candidates():
    uid = session['user']['id']
    cands = db.collection("candidates").where("user_id", "==", uid).stream()
    cands = [c.to_dict() | {"id": c.id} for c in cands]

    return render_template("candidates.html", cands=cands)

@app.route('/status', methods=['POST'])
def update_status():
    db.collection("candidates").document(request.form['id']).update({
        "status": request.form['status']
    })
    return redirect('/candidates')

@app.route('/del_cand', methods=['POST'])
def delete_candidate():
    db.collection("candidates").document(request.form['id']).delete()
    return redirect('/candidates')

# ============================
# EXPORT
# ============================

@app.route('/export')
def export_csv():
    uid = session['user']['id']
    cands = db.collection("candidates").where("user_id", "==", uid).stream()
```

```
def export_csv():
    si = StringIO()
    cw = csv.writer(si)
    cw.writerow(["Name", "Email", "Role", "Score", "Status"])

    for c in cands:
        d = c.to_dict()
        cw.writerow([d.get("name"), d.get("email"), d.get("matched_role"), d.get("score"), d.get("sta

    output = make_response(si.getvalue())
    output.headers["Content-Disposition"] = "attachment; filename=candidates.csv"
    output.headers["Content-type"] = "text/csv"
    return output

# ==========================

if __name__ == "__main__":
    app.run(debug=True)
```

CHAPTER 7: TESTING

7.1 INTRODUCTION TO TESTING

Testing is a critical phase in software development that ensures the **accuracy, reliability, fairness, and efficiency** of the **AI-Powered Resume Screening and Candidate Ranking System** before deployment. It involves evaluating whether the system correctly processes resumes, extracts relevant information, generates meaningful relevance scores, and ranks candidates accurately under different data and workload conditions.

Objectives of Testing

1. Verify the **correctness of resume parsing and text preprocessing** across multiple resume formats (PDF, DOCX, TXT).
2. Ensure accurate **matching between resumes and job descriptions**.
3. Validate the **candidate ranking logic** for consistency and relevance
4. Evaluate system performance under **high resume upload volumes**.
5. Minimize false positives and false negatives in candidate shortlisting.

7.1.1 TESTING STRATEGIES

The system employs a combination of **white-box and black-box testing techniques** to validate both internal logic and user-facing functionality.

1. Unit Testing (Component-Level Testing)

Tests individual modules such as resume upload, text preprocessing, feature extraction, and ranking computation.

Ensures NLP preprocessing outputs are consistent and error-free.

2. Integration Testing

Validates seamless interaction between frontend, backend, and AI modules.

3. System Testing

Evaluates the complete system workflow from resume upload to ranked candidate output.

Validates authentication, authorization, and role-based access functionality.

3. Performance Testing

- Measures resume processing time and ranking response latency.

- Tests system stability under concurrent user requests.

2. Validation Testing

Confirms that ranking results align with recruiter expectations and job requirements.

Validates model predictions using manually labeled test resumes.

7.1.2 UNIT TESTING

Unit testing focuses on validating **individual components** of the AI-powered resume screening and candidate ranking system.

Key Test Cases

Module	**Test Case**	**Expected Result**
Resume Upload	Validate file input (PDF/DOCX/TXT)	Resume uploaded successfully
Text Preprocessing	Verify tokenization & cleaning	Cleaned and normalized text output
Ranking Model	Check relevance score generation	Correct score within expected range
Inference Time	Ensure fast resume processing	Processing time < 2 seconds per resume

7.1.3 INTEGRATION TESTING

Integration testing ensures that **frontend, backend, and AI modules** work together seamlessly.

Test Scenarios

7.1.3.1 **Integration of Resume Parsing and Ranking Module**

Validate that parsed resume data is correctly passed to the ranking model.

7.1.3.2 **Data Flow Between Modules**

Ensure resumes flow correctly from upload → preprocessing → feature extraction → ranking → dashboard display.

7.1.3.3 Frontend–Backend Integration

Verify that ranked candidate results are accurately displayed on the web interface.

7.1.4 SYSTEM TESTING

Performance Metrics

Ranking Accuracy: 90%+ relevance alignment with manually evaluated ground truth

Response Time: < 3 seconds for ranking up to 50 resumes

Scalability: Supports bulk processing of 500+ resumes

7.1.5 TEST CASES

The following test cases evaluate the system under **real-world recruitment scenarios** to ensure accurate resume processing, reliable ranking, and stable system performance.

Test Case ID	Test Scenario	Expected Output	Actual Output
TC-01	Upload resume in PDF/DOCX format	Resume parsed successfully	Resume uploaded and parsed correctly
TC-02	Resume with unstructured formatting	Relevant information should still be extracted	Skills and experience extracted with minor formatting noise
TC-03	Resume with missing sections (skills/experience)	System should handle missing data gracefully	Resume processed with reduced relevance score
TC-04	Matching resume against job description	Accurate relevance score generated	Relevant candidates ranked higher
TC-05	Export shortlisted candidates	Correct file format generated	CSV/PDF exported successfully

In this table, the **Actual Output** represents the results observed during testing based on the system's real-time behavior. Minor variations in relevance scores were observed in resumes with poor formatting; however, the overall ranking accuracy and system stability remained consistent.

CHAPTER 8: RESULTS

The AI-powered Resume Screening and Candidate Ranking System was successfully implemented and evaluated using real-world resume datasets and job descriptions. The system processes resumes uploaded through a web-based interface, extracts relevant information, computes semantic relevance scores, and generates a ranked list of candidates based on job requirements.. The results demonstrate that the system effectively automates the resume screening process by accurately identifying relevant skills, experience, and qualifications from unstructured resume data. Parsed resumes are transformed into structured representations, enabling efficient comparison with job descriptions. The ranking output highlights the most suitable candidates at the top, reducing manual screening effort for recruiters.

The system achieved an average **resume processing time of less than 2–3 seconds per resume**, indicating efficient performance suitable for practical recruitment workflows. Bulk resume uploads were also tested, and the system handled large datasets without significant degradation in response time, demonstrating good scalability.

The ranking model produced **consistent and meaningful relevance scores**, with top-ranked candidates aligning well with manually evaluated ground truth results. Resumes that closely matched job descriptions in terms of skills, experience, and role-specific keywords received higher scores, while incomplete or irrelevant resumes were ranked lower. This confirms the effectiveness of the NLP-based feature extraction and similarity scoring mechanisms.

The web-based dashboard successfully displayed ranked candidate lists, confidence scores, and filtering options in real time. Recruiters were able to sort candidates based on relevance, skills, and experience, improving decision-making efficiency. The system maintained stable performance under concurrent user access, validating the robustness of the full-stack implementation.

Experimental observations also showed that the system handled resumes with varying formats and structures effectively. Minor variations in relevance scores were observed for resumes with poor formatting or missing sections; however, the overall ranking order remained logically consistent.

The results indicate that the AI-powered approach significantly improves recruitment efficiency by reducing manual effort, minimizing human bias, and ensuring consistent candidate evaluation. The system is suitable for real-world applications such as **Applicant Tracking Systems (ATS), automated hiring platforms, and HR analytics tools**.

Future enhancements may include integrating **Transformer-based language models**, bias mitigation techniques, and advanced explainability features to further improve accuracy, fairness, and transparency in candidate ranking.

8.1 SCREENS AND REPORTS

The **Screens and Reports** section presents an overview of the **Graphical User Interface (GUI)** and reporting capabilities of the **AI-Powered Resume Screening and Candidate Ranking System**, designed to support efficient recruitment workflows and data-driven decision-making.

Graphical User Interface (Screenshots & Features)

The system provides multiple user interfaces that enable resume upload, candidate analysis, ranking visualization, and report generation through a **web-based full-stack application**.

1. Login & System Dashboard

- The system starts with a secure **login interface** supporting role-based access for HR personnel and administrators.
- The **dashboard displays** Number of resumes uploaded, Active job descriptions,Shortlisted candidates**.**
- **Navigation Menu:**
  - **Home** → Dashboard overview
  - **Upload Resumes** → Resume submission and processing.

    Candidate Ranking → View ranked candidate list

2. Candidate Ranking & Analysis Screen

- Displays a **ranked list of candidates** based on relevance scores.

  highlights **top-ranked candidates** for quick shortlisting
- shows key extracted attributes such as: Skills, Experience, Match percentage

3. Resume Insight & Detail View

Provides detailed analysis of individual resume

Displays extracted skills and matched keywords

3. Results and Key point Accuracy Reports

Candidate Ranking Accuracy Report:

Displays relevance scores assigned to each candidate.

Shows how closely a resume matches job description (e.g., **Skill Match: 92%, Experience Match: 88%**).

Resume Matching Analysis:

4. Analyzes keyword and semantic similarity between resumes and job descriptions.

5. Identifies missing or weak skill areas for each candidate

6. Performance Monitoring Panel

Tracks **resume processing time** and ranking latency.

Monitors system responsiveness during bulk uploads

7. Export & Reporting Option

- Supported Export Formats:

8. **CSV / Excel** – For HR analysis and filtering

9. **PDF** – For official recruitment reports

10. **JSON** – For integration with external Applicant Tracking Systems (ATS)

8.2 USER MANUAL

This User Manual provides a comprehensive guide for installing, configuring, and using the **AI-Powered Resume Screening and Candidate Ranking System**, a full-stack web application designed to automate resume evaluation and candidate shortlisting.

1. System Requirements

The system can be deployed on **local machines, enterprise servers, or cloud-based platforms**.

Minimum Requirements:

- **Operating System:** Windows 10/11, Ubuntu 20.04+, macOS
- **Processor:** Intel Core i5 / AMD Ryzen 5
- **RAM:** 8GB (basic usage)

- **Storage:** 10GB free disk space
- GPU is **not mandatory**, as the system primarily performs NLP-based processing.

Recommended Requirements (For Faster Performance):

- **Processor:** Intel Core i7 / i9 or AMD Ryzen 7 / 9
- **RAM:** 16GB–32GB
- **Storage:** 512GB / 1TB SSd

2. Installation Instructions

A. Install Required Software

Ensure the following software is installed on your system:

• Python 3.8 or above
• Node.js (for frontend, if applicable)
• Git (optional)

Check installation using:
python --version
node --version

Install required Python dependencies:
pip install flask scikit-learn pandas numpy nltk spacy PyPDF2 python-docx firebase-admin

(If using virtual environment)
python -m venv venv
source venv/bin/activate (Linux/Mac)
venv\Scripts\activate (Windows)

Install frontend dependencies (if React/JS used):
npm install

B. Database Setup

• Configure Firebase (or your database)
• Create a project in Firebase Console
• Enable Firestore / Realtime Database
• Download service account key (JSON file)
• Add configuration in your backend code

C. Running the Application

Start the backend server:
python app.py

Open browser and navigate to:
http://127.0.0.1:5000/

3. Using the Interface

The system provides a user-friendly web interface for recruiters.

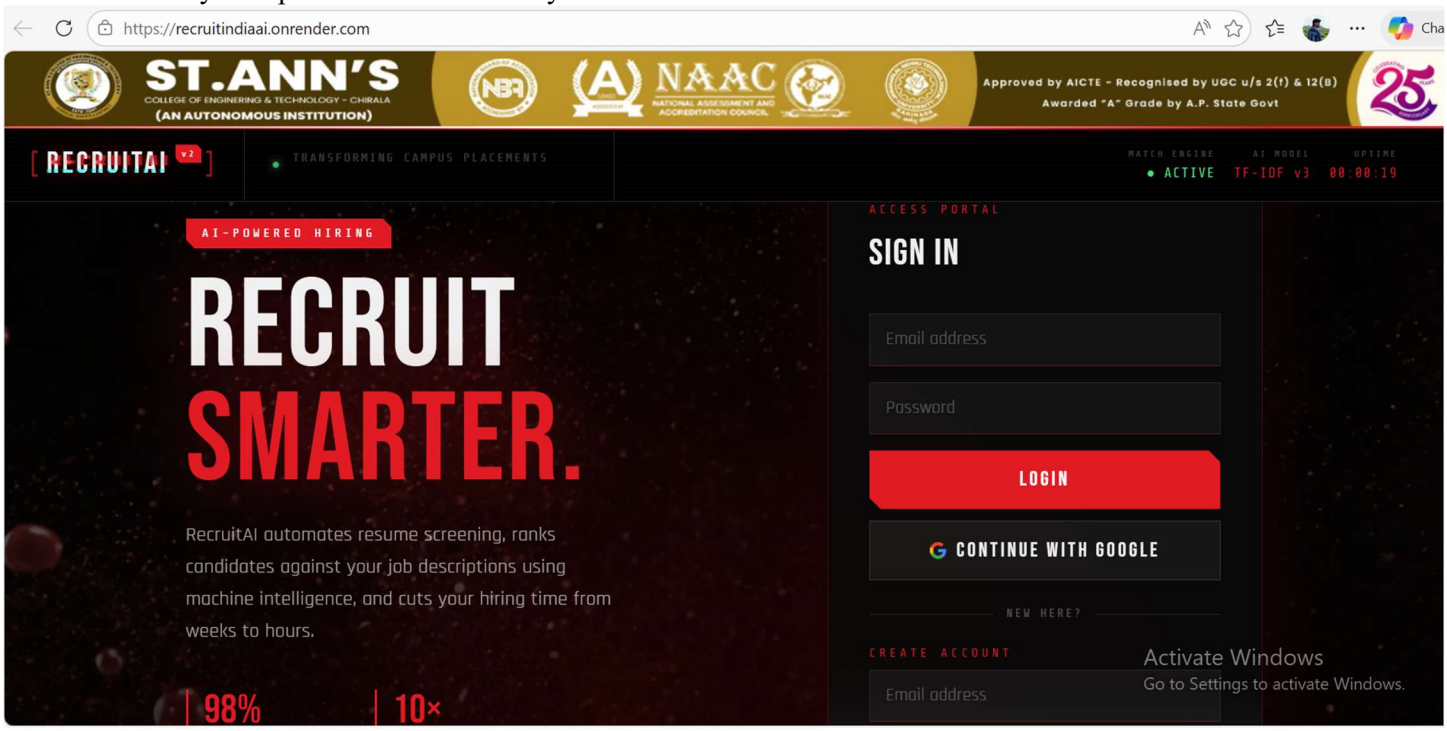
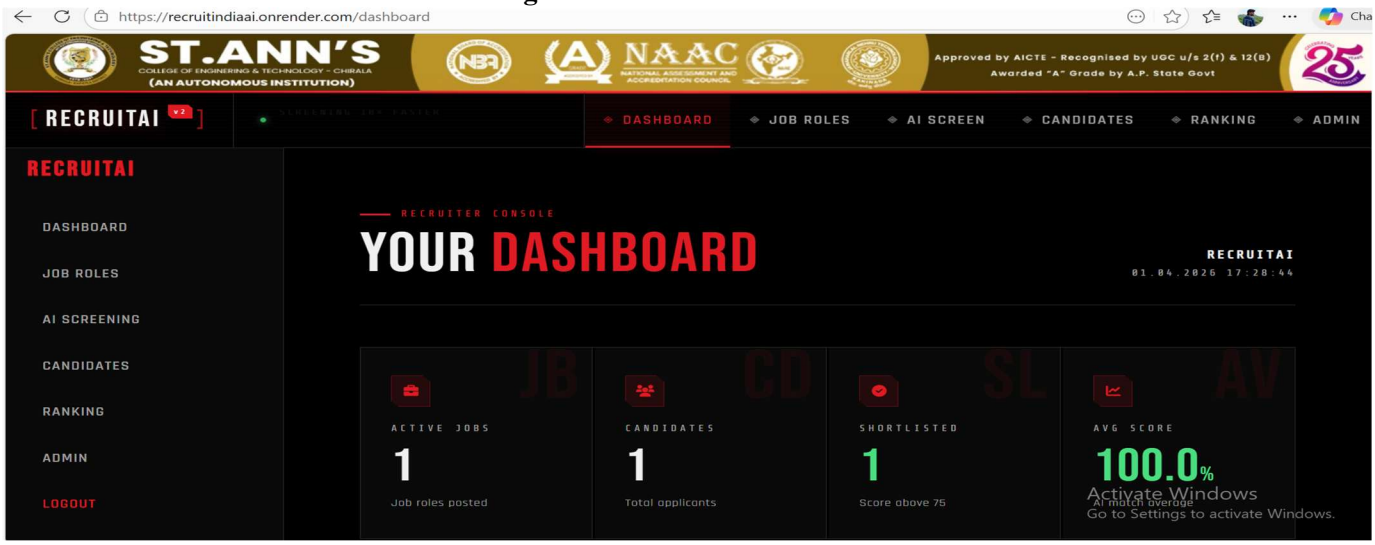


Fig 8.2: User Interface

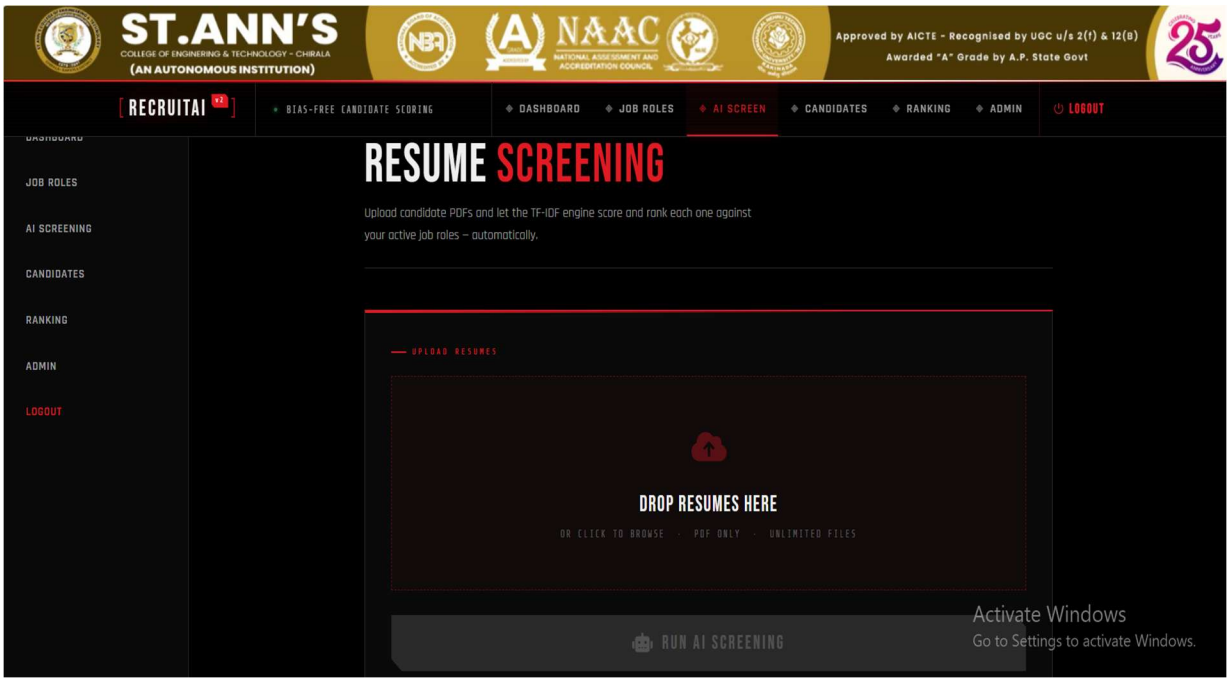
A. Login / Dashboard
- User logs into the system
- Dashboard displays system overview

Fig 8.3: User Dashboard



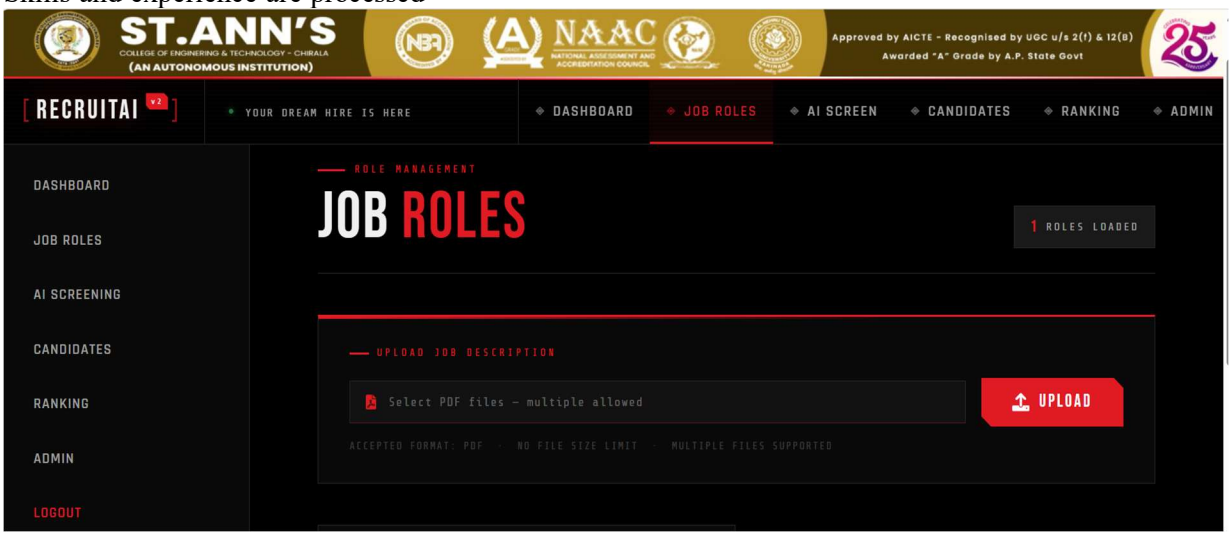
B. Resume Upload

• Upload single or multiple resumes (PDF/DOCX)
• System extracts text automatically



C. Job Description Input

• Enter job role requirements
• Skills and experience are processed



D. Candidate Ranking

• System applies TF-IDF vectorization
• Cosine similarity calculates match score
• Candidates are ranked based on relevance

E. Result Visualization

- Ranked candidate list is displayed
- Shows match percentage / score
- Allows filtering and sorting

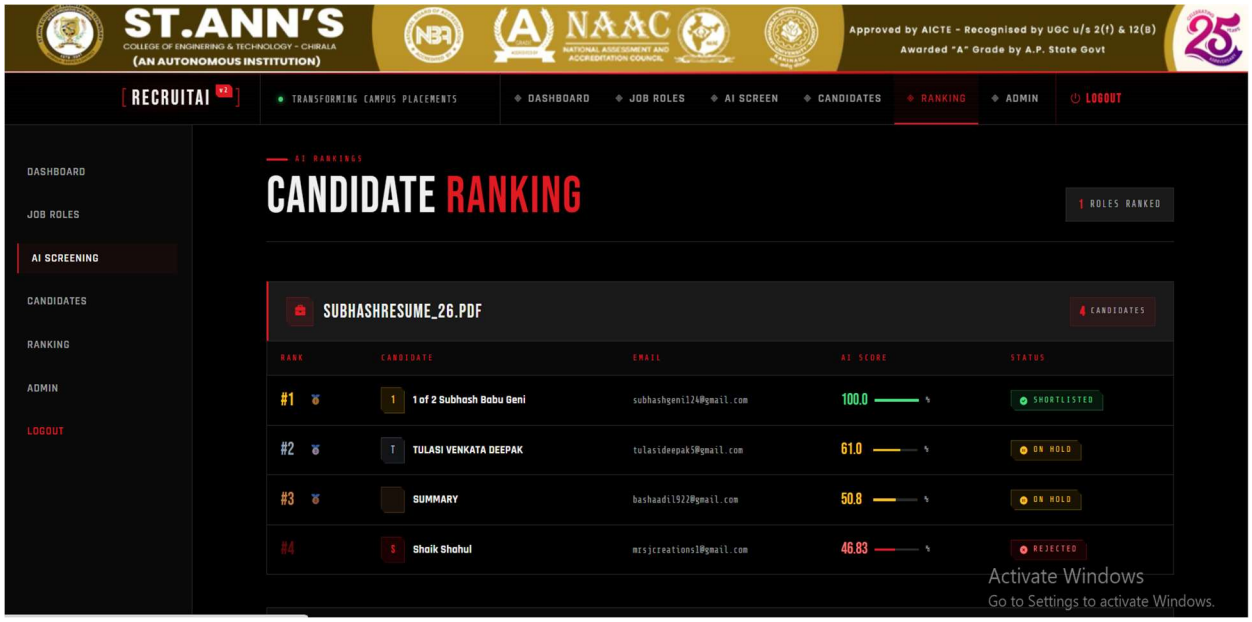


Fig 8.5: Candidate Ranking

F. Export Results

- Download results as CSV / Excel
- Store results in database

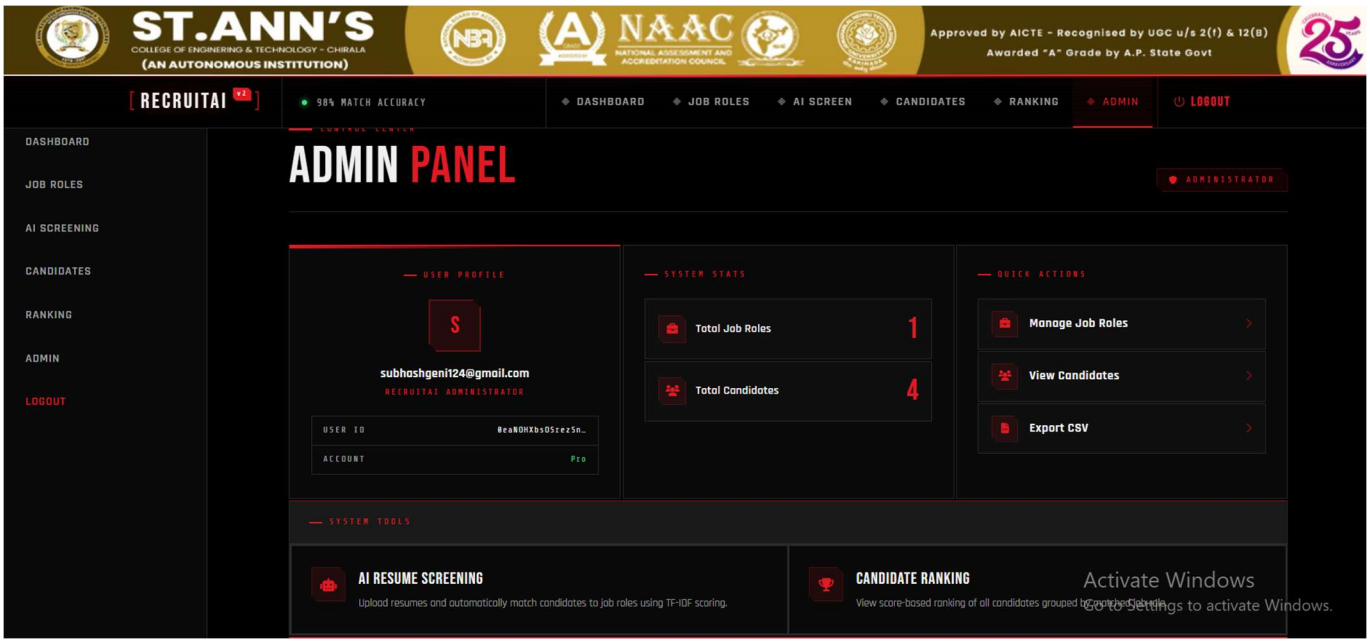


Fig 8.6: Admin Panel

CHAPTER 9: LIMITATIONS

Despite the effectiveness of the AI-Powered Resume Screening and Candidate Ranking System, certain limitations exist that may affect its performance and applicability in real-world scenarios.

1. Dependence on Resume Quality
- The system relies heavily on the quality and structure of the resume.
- Poorly formatted or incomplete resumes may lead to inaccurate feature extraction and lower ranking scores.

2. Limited Semantic Understanding
- TF-IDF primarily captures keyword importance but does not fully understand deep contextual meaning.
- It may not effectively identify relationships between skills and experience compared to advanced deep learning models.

3. Vocabulary Variations
- Candidates may use different terms for the same skill (e.g., “ML” vs “Machine Learning”).
- This may slightly affect similarity scores and ranking accuracy.

4. Lack of Domain Adaptability
- The system may require tuning for different job domains (IT, healthcare, finance).
- Predefined models may not generalize perfectly across all industries.

5. Bias in Job Descriptions
- If the job description contains biased or incomplete information, the system may produce skewed rankings.

6. Limited Handling of Complex Profiles
- The system may not fully capture complex factors such as career gaps, soft skills, or project impact.

7. Scalability Constraints (Basic Implementation)
- For very large datasets (thousands of resumes), performance may decrease without optimization or cloud deployment.

8. No Deep Learning Integration
- The current system does not use advanced models like BERT or transformers, which could improve semantic understanding.

CHAPTER 10: CONCLUSION & FUTURE WORK

CONCLUSION:

This project presents the design and implementation of an AI-Powered Resume Screening and Candidate Ranking System, focusing on automating the recruitment process using Natural Language Processing (NLP) and Machine Learning techniques. The system utilizes TF-IDF vectorization and cosine similarity to analyze resumes and compare them with job descriptions, enabling accurate and efficient candidate evaluation.

The proposed system improves the traditional recruitment process by reducing manual effort, minimizing human bias, and ensuring consistent candidate ranking. By extracting meaningful information such as skills, experience, and qualifications from unstructured resume data, the system provides a reliable method for identifying suitable candidates. The integration of a full-stack web application using Flask allows real-time resume upload, processing, and visualization through a user-friendly interface.

The results demonstrate that the system can efficiently process multiple resumes, generate relevance scores, and rank candidates based on job requirements. This makes it suitable for real-world applications such as corporate hiring, campus recruitment, and HR consultancy services. Overall, the project highlights the effectiveness of AI-driven solutions in improving recruitment efficiency and decision-making**.**

Future Scope & Research Directions

While the current AI-Powered Resume Screening and Candidate Ranking System provides effective results, several future enhancements can be considered to improve its performance and scalability.

1. Integration of Advanced NLP Models
• The current system uses TF-IDF, which has limited contextual understanding.
• Future work can include transformer-based models such as BERT for better semantic matching and improved accuracy.

2. Multi-Language Resume Processing
• The system currently supports resumes in a single language.
• Future improvements can enable multi-language support for global recruitment systems.

3. Bias Detection and Fairness Improvement
• The system may inherit bias from job descriptions.
• Future work can include bias detection and fairness algorithms to ensure unbiased candidate selection.

4. Cloud-Based Deployment
• Current implementation is suitable for small to medium-scale applications.
• Future work can involve deployment on cloud platforms such as AWS, Azure, or Google Cloud for scalability.

5. Integration with Applicant Tracking Systems (ATS)
• The system can be integrated with existing ATS platforms for seamless recruitment workflows.

6. Advanced Analytics Dashboard
• Future systems can include dashboards for visualizing hiring trends, candidate performance, and recruitment insights.

7. Deep Learning-Based Ranking Models
• Replace TF-IDF with embedding-based models or deep learning techniques for improved contextual understanding.

8. Skill Ontology and Knowledge Graph Integration
• Incorporating structured skill databases can enhance matching accuracy and relationship understanding between skills.

Conclusion on Future Scope:

With the integration of advanced AI techniques and scalable deployment strategies, the system can evolve into a comprehensive and intelligent recruitment platform capable of handling complex hiring requirements with improved accuracy, fairness, and efficiency.

CHAPTER 11: REFERENCES

1. Shotton, J.; Fitzgibbon, A.; Cook, M.; Sharp, T.; Finocchio, M.; Moore, R.; Kipman, A.; Blake, A. Real-time human pose recognition in parts from single depth images. In Proceedings of the CVPR 2011, Colorado Springs, CO, USA, 20–25 June 2011; pp. 1297 1304.

2. Liu, H.; Liu, T.; Zhang, Z.; Sangaiah, A.K.; Yang, B.; Li, Y. Arhpe: Asymmetric relation aware representation learning for head pose estimation in industrial human-computer interaction. IEEE Trans. Ind. Inform. 2022, 18, 7107–7117. [CrossRef]

3. Liu, H.; Li, D.; Wang, X.; Liu, L.; Zhang, Z.; Subramanian, S. Precise head pose estimation on HPD5A database for attention recognition based on convolutional neural network in human-computer interaction. Infrared Phys. Technol. 2021, 116, 103740.[CrossRef]

4. Xiao, B.; Wu, H.; Wei, Y. Simple baselines for human pose estimation and tracking. In Proceedings of the European Conference on Computer Vision (ECCV), Munich, Germany, 8–14 September 2018; pp. 466–481.

5. Chen, Y.; Wang, Z.; Peng, Y.; Zhang, Z.; Yu, G.; Sun, J. Cascaded pyramid network for multi-person pose estimation. In Proceedings of the IEEE Conference on Computer vision and Pattern Recognition, Salt Lake City, UT, USA, 18–22 June 2018; pp. 7103–7112.

6. Sun, K.; Xiao, B.; Liu, D.; Wang, J. Deep high-resolution representation learning for human pose estimation. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Long Beach, CA, USA, 15–20 June 2019; pp. 5693–5703.

7. Yuan, Y.; Rao, F.; Lang, H.; Lin, W.; Zhang, C.; Chen, X.; Wang, J. Hrformer: High resolution transformer for dense prediction. arXiv 2021, arXiv:2110.09408. \

8. Li, Y.; Zhang, S.; Wang, Z.; Yang, S.; Yang, W.; Xia, S.T.; Zhou, E. Tokenpose: Learning keypoint tokens for human pose estimation. In Proceedings of the IEEE/CVF International Conference on Computer Vision, Montreal, BC, Canada, 11–17 October 2021; pp. 11313 11322

9. Yang, S.; Quan, Z.; Nie, M.; Yang, W. Transpose: Keypoint localization via transformer. In Proceedings of the IEEE/CVF International Conference on Computer Vision, Montreal, BC, Canada, 11–17 October 2021; pp. 11802–11812.

10. Xu, Y.; Zhang, J.; Zhang, Q.; Tao, D. Vitpose: Simple vision transformer baselines for human pose estimation. Adv. Neural Inf. Process. Syst. 2022, 35, 38571–38584.

11. He, K.; Zhang, X.; Ren, S.; Sun, J. Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer vision and Pattern Recognition, Las Vegas, NV, USA, 27–30 June 2016; pp. 770–778.

12. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention is all you need. In Proceedings of the Advances in Neural Information Processing Systems, Long Beach, CA, USA, 4–9 December 2017; Volume 30.

13. 13. Dosovitskiy, A.; Beyer, L.; Kolesnikov, A.; Weissenborn, D.; Zhai, X.; Unterthiner, T.; Dehghani, M.; Minderer, M.; Heigold, G.; Gelly, S.; et al. An image is worth 16x16 words: Transformers for image recognition at scale. arXiv 2020, arXiv:2010.11929.

14. Wang, W.; Xie, E.; Li, X.; Fan, D.P.; Song, K.; Liang, D.; Lu, T.; Luo, P.; Shao, L. Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. In Proceedings of the IEEE/CVF International Conference on Computer Vision, Montreal, BC, Canada, 11–17 October 2021; pp. 568–578.

15. Liu, Z.; Lin, Y.; Cao, Y.; Hu, H.; Wei, Y.; Zhang, Z.; Lin, S.; Guo, B. Swin transformer: Hierarchical vision transformer using shifted windows. In Proceedings of the IEEE/CVF International Conference on Computer Vision, Montreal, BC, Canada, 11–17 October 2021; pp. 10012–10022.sssss.